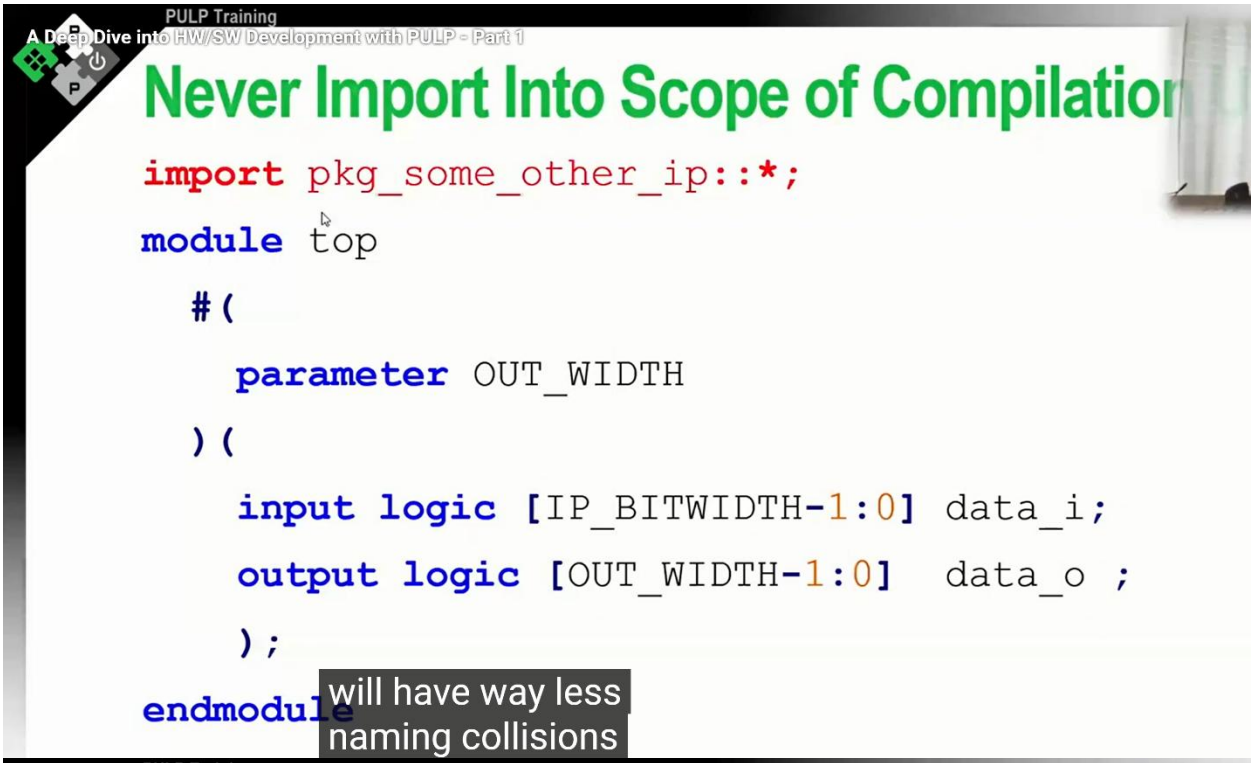
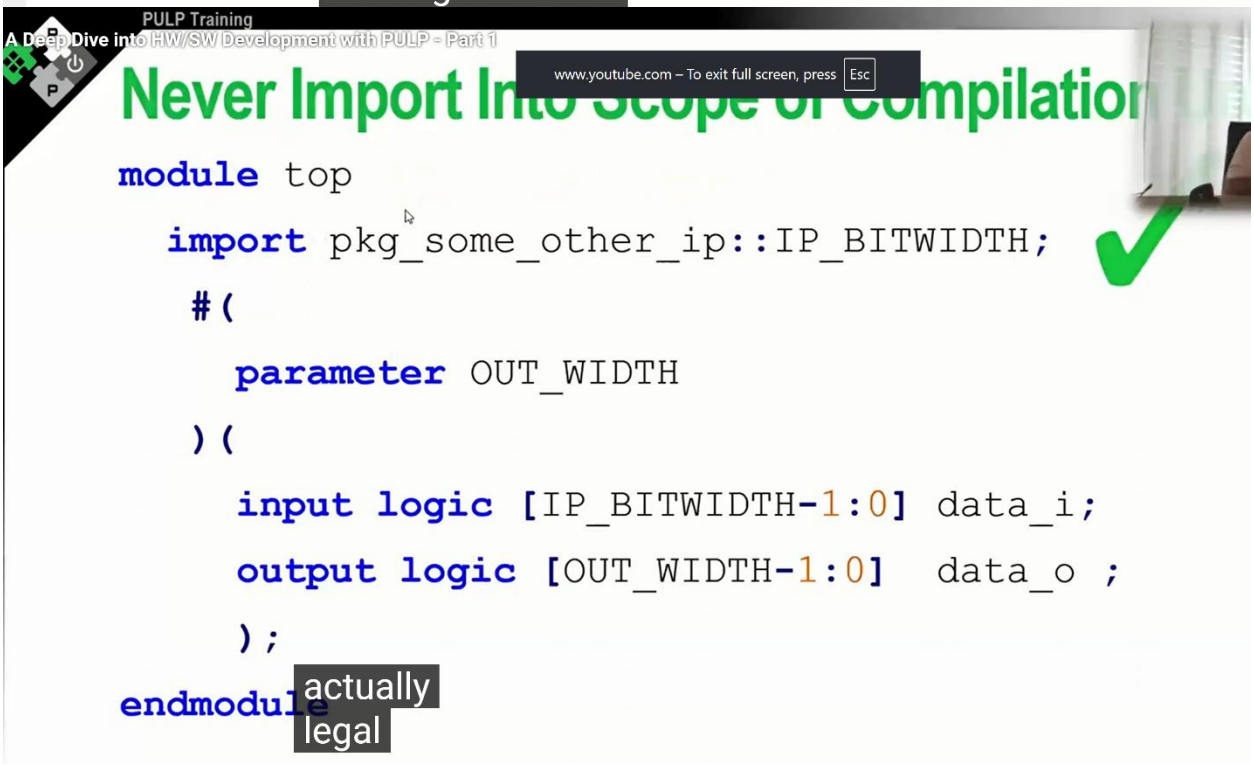
A Deep Dive into HW/SW Development with PULP - Part 1





The core problem he's warning about

🔴 "Never import into scope of compilation"

This line is **about where you put `import pkg::*;` in SystemVerilog**, and how **different tools define a "compilation unit."**

The danger is **name collisions** that depend on the simulator/synthesizer, not your code.

What is a *compilation unit* in SystemVerilog?

A **compilation unit** is the scope in which:

- packages
- typedefs
- parameters
- functions
- variables

exist **without qualification**.

Tool behavior differs 👇

☑ Some tools (good behavior)

- Each `.sv` file = **its own compilation unit**
- Imports at top of file only affect that file

❌ Other tools (Vivado default, some synthesis flows)

- **ALL `.sv` files share ONE global compilation unit**
- Any `import pkg::*;` at the top of *any* file pollutes *everything*

➡ This is where things explode.

The "atrocity" example (what NOT to do)

```
import pkg_some_other_ip::*;

module top #(parameter OUT_WIDTH)(
```

```
  input  logic [IP_BITWIDTH-1:0] data_i,
  output logic [OUT_WIDTH-1:0]    data_o
);
endmodule
```

Why this is dangerous

- `IP_BITWIDTH` is now **globally visible**
- If *another package* defines `IP_BITWIDTH`, you get:
  - ambiguous references
  - silent overrides
  - tool-dependent behavior 😬

In a **huge SoC like PULP**, this is a nightmare.

☑ The correct (recommended) way

```
module top
  import pkg_some_other_ip::IP_BITWIDTH;
#(
  parameter OUT_WIDTH
)(
  input  logic [IP_BITWIDTH-1:0] data_i,
  output logic [OUT_WIDTH-1:0]    data_o
);
endmodule
```

Why this is GOOD

✔ The import is **scoped to the module only**
✔ No global namespace pollution
✔ Same behavior across tools (Vivado, Questa, Verilator, etc.)
✔ Much fewer naming collisions
✔ Scales to large projects like **PULP SoC**

This is exactly why the tutor says:

“If everybody does that we would have way less troubles trying to compile PULP SoC with tools that have global compilation units.”

Even better: explicit qualification (safest option)

When practical, this is **bulletproof**:

```
input logic [pkg_some_other_ip::IP_BITWIDTH-1:0] data_i;
```

✔ Zero ambiguity
✔ Slightly more verbose
✔ Used heavily in high-quality RTL (including parts of PULP)

Why this matters *specifically* for PULP

PULP has:

- dozens of IP blocks
- many reused packages
- many common names (`WIDTH`, `ADDR_WIDTH`, `DATA_WIDTH`, `DEPTH`, etc.)
- multiple toolchains (open-source + vendor tools)

If packages are imported globally:

- **compilation order matters**
- **tool choice matters**
- bugs appear/disappear randomly

That's why PULP coding style is **very strict** about this.

Practical rule of thumb (memorize this)

❌ Avoid

```
import my_pkg::*;
```

outside of a module/interface/program

☑ Prefer

```
module foo
  import my_pkg::SOME_TYPE;
```

☑ Best (when clarity matters)

```
my_pkg::SOME_TYPE x;
```

One-liner summary

Packages should be imported where they are used, not where they are compiled.

That single rule saves **weeks of debugging** in SoC-scale projects.

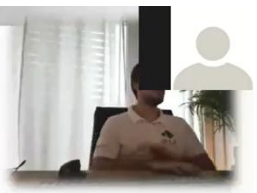
PULP Training

A Deep Dive into HW/SW Development with PULP - Part 1

Parameters/Constants

```
module my_ip
  #(
    parameter WIDTH = 32,
    parameter BE_WIDTH = WIDTH/8 //could be changed
  )(
    input logic [WIDTH-1:0]    data_i;
    input logic [BE_WIDTH-1:0] be_i;
    );
 parameter MY_CONSTANT = 42; //I'm not a constant :-(
endmodule
```

12:08 / 5:08:07



PULP Training

A Deep Dive into HW/SW Development with PULP - Part 1

Parameters/Constants

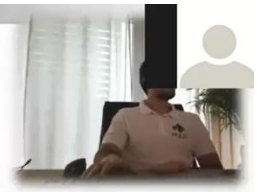
```
module my_ip
  #(
    parameter WIDTH = 32,
    parameter BE_WIDTH = WIDTH/8 //could be changed
  )(
    input logic [WIDTH-1:0]    data_i;
    input logic [BE_WIDTH-1:0] be_i;
    );
 parameter MY_CONSTANT = 42; //I'm not a constant :-(
endmodule
```

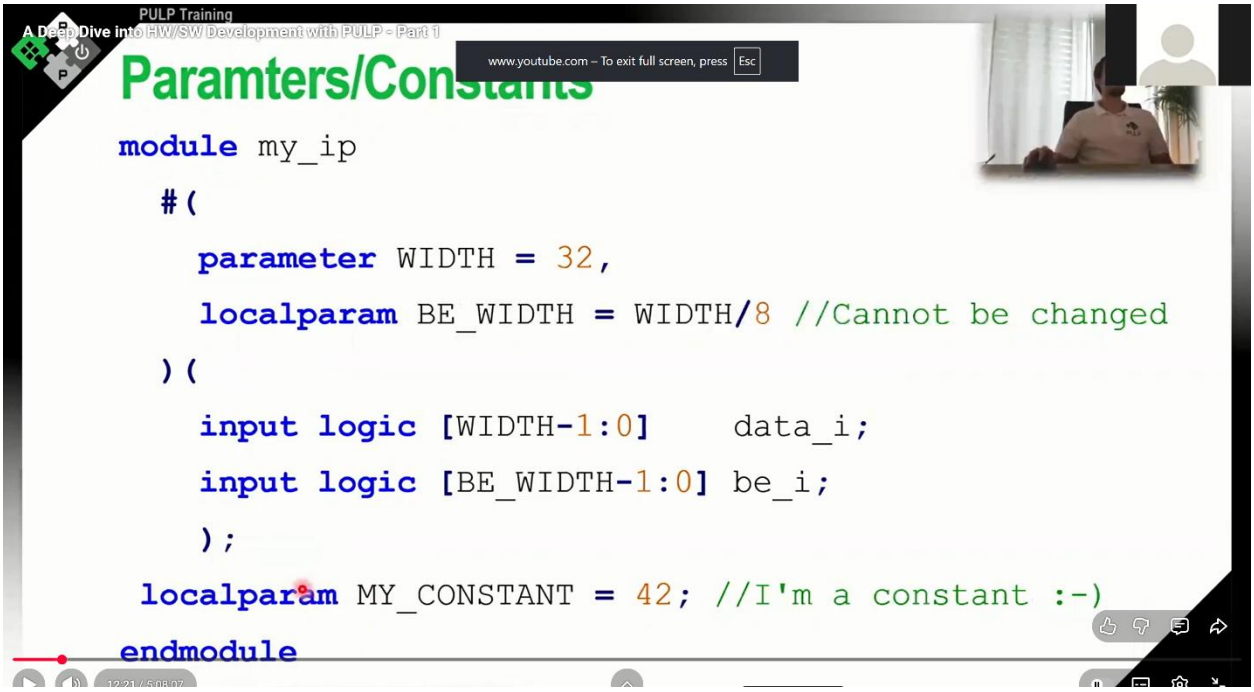
X If you want a (derived) constant, use localparam

X

12:16 / 5:08:07

17.12.2020





---

1 The core idea (one sentence)

`parameter` is part of the module's public API; `localparam` is an internal constant.

That's it. Everything else flows from this.

---

2 What goes wrong in the "bad" example

❌ Original code (problematic)

```
module my_ip #(
  parameter WIDTH    = 32,
  parameter BE_WIDTH = WIDTH/8   // derived, but still a parameter ❌
)(
  input  logic [WIDTH-1:0]     data_i,
  input  logic [BE_WIDTH-1:0]  be_i
);

parameter MY_CONSTANT = 42; // ❌ not really constant
endmodule
```

Why this is bad

❌ Problem 1: `BE_WIDTH` can be overridden

Even though it is *derived*, it is still a **parameter**, so this is legal:

```
my_ip #(
  .WIDTH(64),
  .BE_WIDTH(3)   // 💥 breaks your design assumptions
) u_my_ip ();
```

Now:

- `WIDTH = 64`
- `BE_WIDTH = 3`
- Byte-enable no longer matches data width
- No syntax error
- Bug shows up **at runtime or synthesis**

This is the *worst* kind of bug.

❌ Problem 2: `MY_CONSTANT` is not a constant

```
parameter MY_CONSTANT = 42;
```

Someone can do this:

```
my_ip #(.MY_CONSTANT(13)) u_my_ip();
```

So the comment *"I'm not a constant :("* is 100% accurate.

---

3 The correct pattern (what PULP enforces)

☑ Fixed version

```
module my_ip #(
  parameter WIDTH = 32
)(
  input  logic [WIDTH-1:0] data_i,
  input  logic [BE_WIDTH-1:0] be_i
);

localparam BE_WIDTH    = WIDTH / 8;   // derived, cannot be overridden
localparam MY_CONSTANT = 42;          // truly constant

endmodule
```

---

4 Why `localparam` is the right tool

☑ `localparam` properties

Property	`parameter`	`localparam`
Can be overridden at instantiation	✅ Yes	❌ No
Visible outside module	✅ Yes	❌ No
Can be derived from parameters	✅ Yes	✅ Yes
Safe for internal invariants	❌ No	✅ Yes
Should be used for constants	❌ No	✅ Yes

---

5 Mental model (very important)

Think of your module like a **C++ class**:

```
class MyIP {
public:
  int WIDTH;        // parameter
private:
  int BE_WIDTH;     // localparam
};
```

- `parameter` → **public API**
- `localparam` → **private implementation detail**

If it must **never change**, it must be `localparam`.

---

6 Why this matters a LOT in PULP

PULP designs are:

- deeply hierarchical
- heavily parameterized

- reused across chips (PULPissimo, GAP, clusters, accelerators)

If derived values are exposed as parameters:

- top-level SoC integration can silently break IPs
- different tools override parameters differently
- debugging becomes nightmare fuel

This is why PULP coding guidelines are **very strict** about this.

---

7 Rule of thumb (memorize this)

🔒 **If a value is derived →** `localparam`
🔓 **If a value is configuration →** `parameter`

Examples:

```
parameter DATA_WIDTH = 32;          // configurable → OK
localparam ADDR_LSB  = $clog2(32);  // derived → MUST be localparam
```

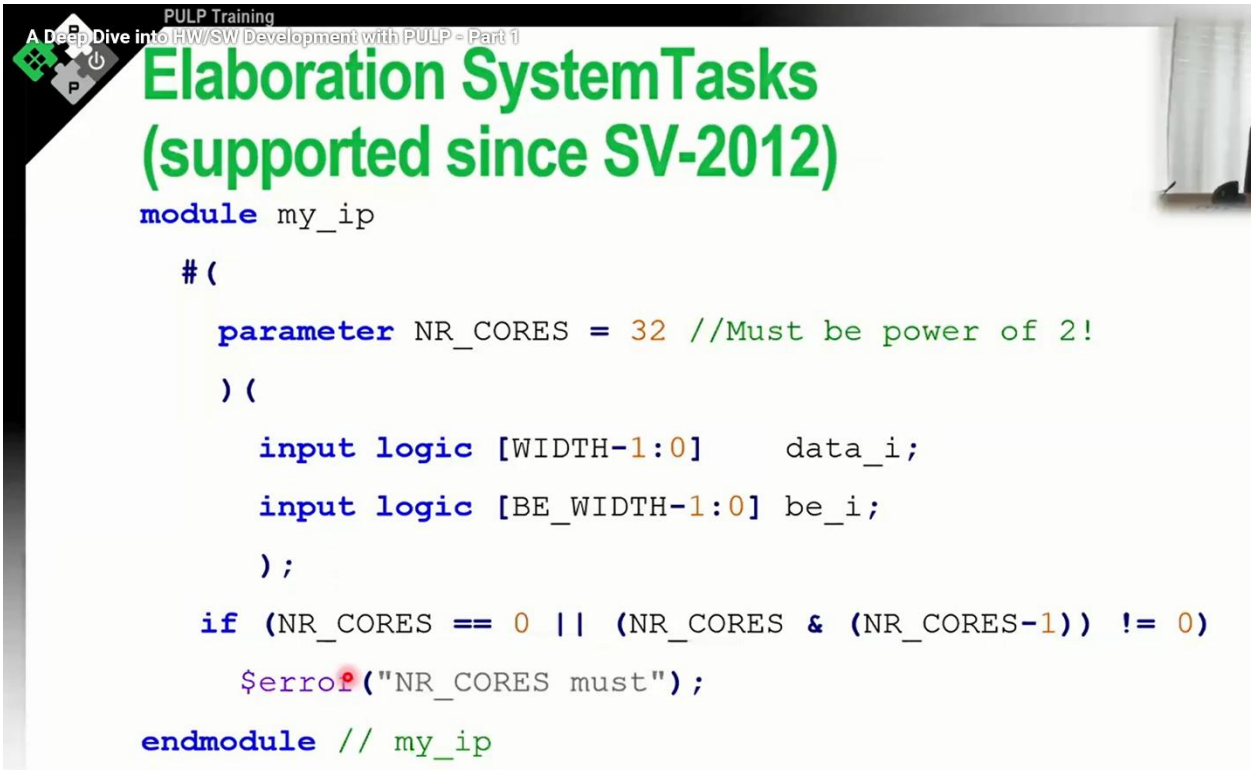
---

8 Why the tutor is so harsh about it

Because:

- simulators won't warn you
- synthesis won't warn you
- lint *might* warn you
- silicon will fail quietly 😄

This slide is basically:
"I've been burned by this before. Please don't repeat it."



---

1 What “elaboration” actually means (crucial concept)

SystemVerilog has **phases**:

1. **Parsing** – syntax check
2. **Elaboration** – parameters resolved, generate blocks expanded, hierarchy built
3. **Simulation / Synthesis** – time-based execution or hardware mapping

👉 **Elaboration happens before simulation *and* before synthesis.**

So anything checked **during elaboration**:

- fails **early**
- fails **deterministically**
- fails in **both simulation and synthesis**

That’s exactly what we want.

---

2 What the slide is doing

Code on the slide (simplified)

```
module my_ip #(
  parameter NR_CORES = 32  // Must be power of 2!
)(
  input logic [WIDTH-1:0] data_i,
  input logic [BE_WIDTH-1:0] be_i
);

if (NR_CORES == 0 || (NR_CORES & (NR_CORES-1)) != 0)
  $error("NR_CORES must be power of 2");

endmodule
```

---

3 Why this check is mathematically correct

This is the classic **power-of-two test**:

```
x is power of 2  ⇔  x != 0 AND (x & (x - 1)) == 0
```

Examples:

NR_CORES	Binary	NR_CORES & (NR_CORES-1)	Result
1	0001	0000	✅ OK
2	0010	0000	✅ OK
4	0100	0000	✅ OK
3	0011	0010	❌
6	0110	0100	❌

This matters because:

- interconnect trees
- arbitration
- address decoding

often rely on powers of two.

4️⃣ Why this is better than comments or asserts

❌ Comment-only protection (bad)

```
parameter NR_CORES = 32; // Must be power of 2!
```

- Humans forget
- Tools don’t care
- Bugs slip through

❌ Runtime `assert` (not ideal)

```
always_comb assert (NR_CORES is power of 2);
```

- Simulator-only
- Can be disabled
- Synthesis may ignore it

☑️ Elaboration-time `$error` (best)

```
if (bad_parameter)
  $error("...");
```

This:

- runs during elaboration
- breaks simulation **and synthesis**
- fails before any logic is built

This is exactly what the tutor means by:

“extra protection where it won't be only checked by simulator but will also be checked by the synthesizer”

5️⃣ Why this works without `generate`

Key subtlety:
This `if` is **not procedural**.

At elaboration time:

- `NR_CORES` is already a constant
- the `if` condition is evaluated once
- `$error` executes immediately if needed

No clock. No time. No logic.

---

6 Why this is critical in PULP-scale designs

In PULP:

- parameters propagate across **dozens of modules**
- clusters, cores, DMA, interconnects depend on them
- wrong values break arbitration, banking, address mapping

If you let a bad parameter slip:

- the design might still synthesize
- but behave incorrectly in silicon

So PULP philosophy is:

Fail fast. Fail loud. Fail before hardware exists.

---

7 Why SV-2012 matters here

Older Verilog:

- had very limited elaboration checking
- relied on simulation hacks

SystemVerilog (SV-2012+):

- allows clean elaboration checks
- tools agree on semantics
- synthesis supports it

That’s why the slide explicitly says:

“supported since SV-2012”

---

8 Best-practice pattern (memorize this)

☑ Parameter validation template

```
module foo #(
  parameter int N = 8
)();

  if (N <= 0)
    $error("N must be > 0");

  if ((N & (N-1)) != 0) // power of 2 check
    $error("N must be power of 2");

endmodule
```

Use this whenever:

- sizes
- counts
- widths
- bank numbers
- interleaving factors

are involved.

---

9 Mental model

Think of elaboration checks as **compile-time `static_assert` in C++**:

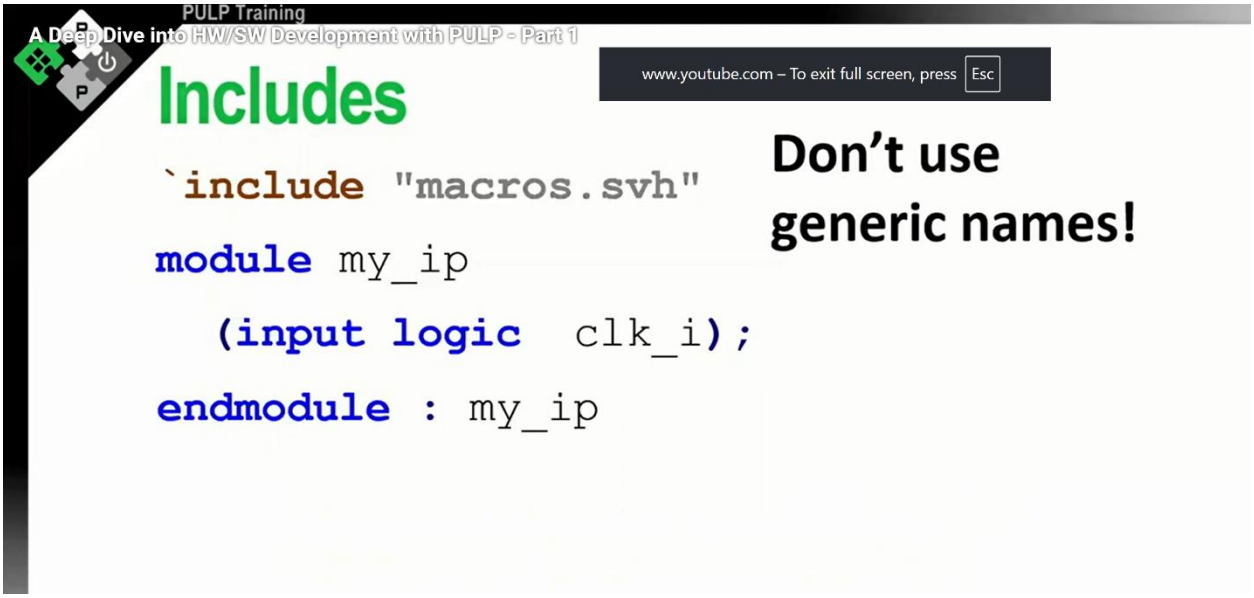
```
static_assert(is_power_of_two(N), "N must be power of 2");
```

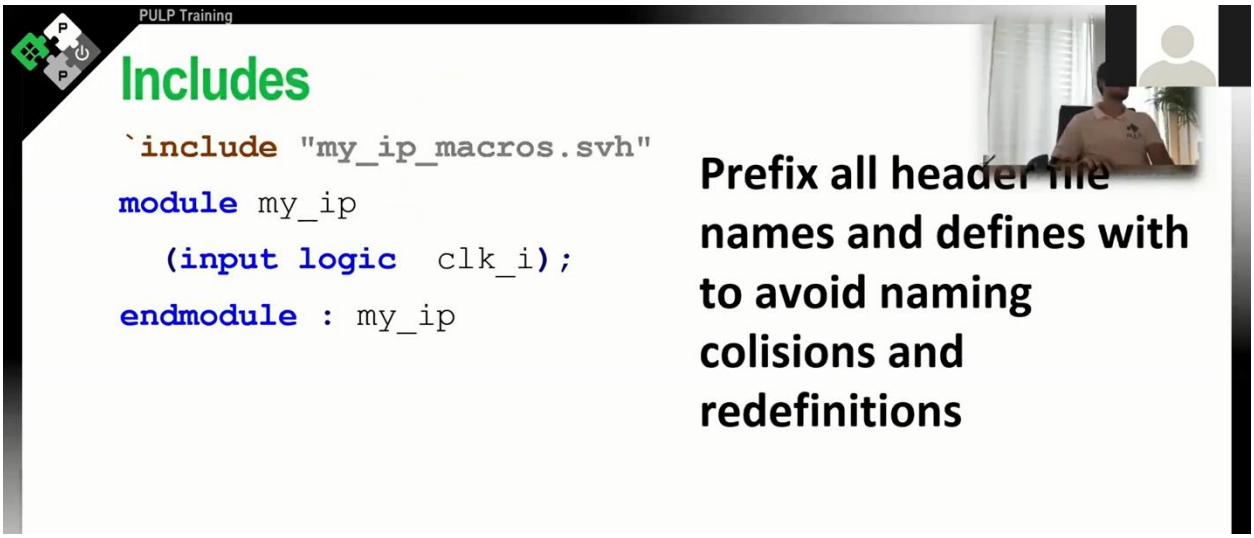
Same intent. Same value.

⬅ END Big takeaway

- Elaboration system tasks are **design contracts**

- They protect **integration**, not just simulation
- They are a cornerstone of **PULP's robustness**





---

1 What `include` really does (this is the root of the problem)

```
`include "macros.svh"
```

This is **not** like importing a module or a package.

👉 It is a **textual copy-paste** done by the preprocessor **before compilation**.

So if `macros.svh` contains:

```
`define ENABLE_DEBUG
`define WIDTH 32
```

Then every file that includes it will literally have those lines pasted into it.

⚠️ That means:

- Macros are **global**
- There is **no scope**
- Order matters
- Redefinitions are easy and dangerous

---

2 Why “Don’t use generic names”

Bad example (`macros.svh`):

```
`define WIDTH 32
`define RESET 1'b0
```

Now imagine:

- IP A includes `macros.svh`
- IP B includes *another* `macros.svh`
- Both define `WIDTH`

💥 Result:

- Redefinition warnings
- Silent overrides (even worse)
- Tool-dependent behavior
- Bugs that appear only with certain compile orders

This is why he says:

“Don’t use generic names!”

Because **macros live in a single global namespace**.

---

3 The correct rule: prefix EVERYTHING in headers

Instead of this ❌:

```
`define WIDTH 32
```

Do this ☑:

```
`define MY_IP_WIDTH 32
```

Or in PULP style:

```
`define PULP_MY_IP_WIDTH 32
```

That's why the slide shows:

```
`include "my_ip_macros.svh"
```

and **not**:

```
`include "macros.svh"
```

The rule is:

Header file name AND macro names must be IP-unique

4️⃣ Why this matters *extra* in PULP

PULP has:

- Many IP blocks
- Many vendors
- Many tools (Verilator, Questa, VCS, Vivado, etc.)
- Often a **global compilation unit**

So if two IPs both do:

```
`define DEBUG
```

You're done. No namespace. No protection.

That's why PULP:

- Avoids macros when possible
- Uses packages instead
- When macros are needed → **strict prefixing**

5️⃣ What he means by “naming collisions and redefinitions”

Collision

Two macros with the same name:

```
`define ENABLE 1
`define ENABLE 0   // collision
```

Redefinition

Same macro defined again later:

```
`define FIFO_DEPTH 8
...
`define FIFO_DEPTH 16  // silently overrides in some tools
```

Some tools:

- Warn
- Error
- Or quietly accept it 😬

That’s why he’s paranoid about it — and he’s right.

6️⃣ Best-practice checklist (PULP-style)

☑ DO

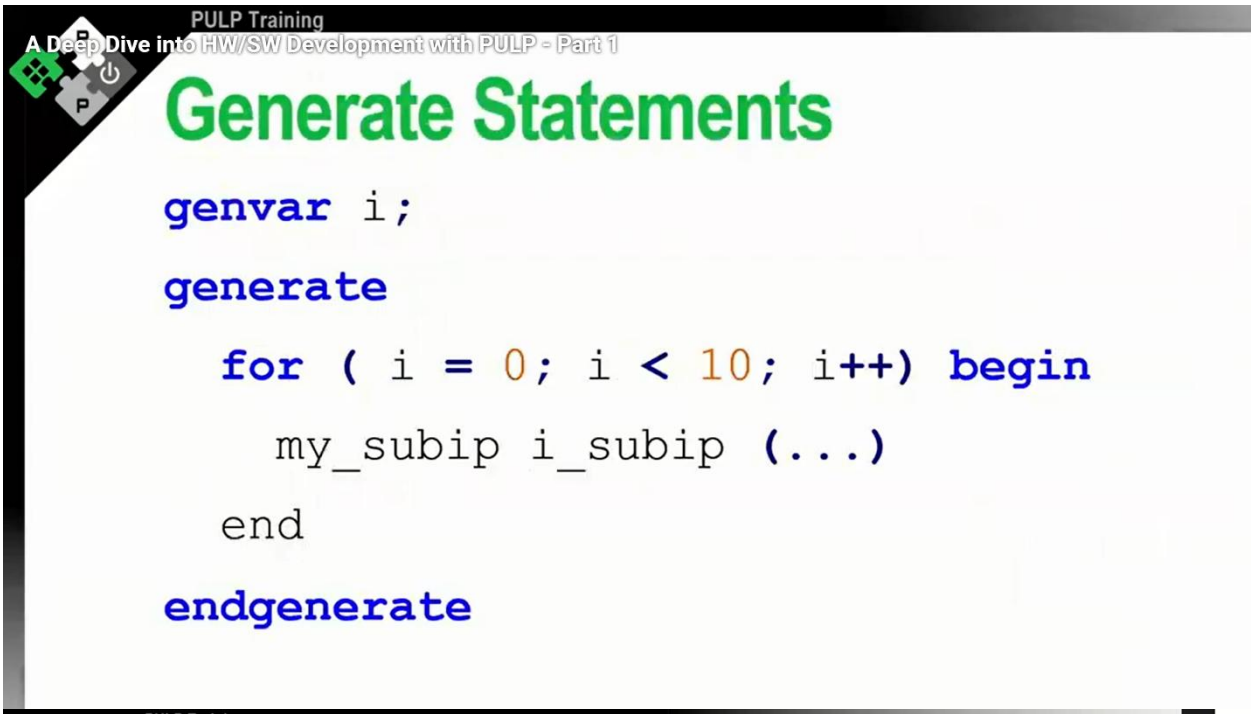
- Prefix header filenames
- Prefix **every macro**
- Use packages instead of macros when possible
- Keep `include` files small and simple

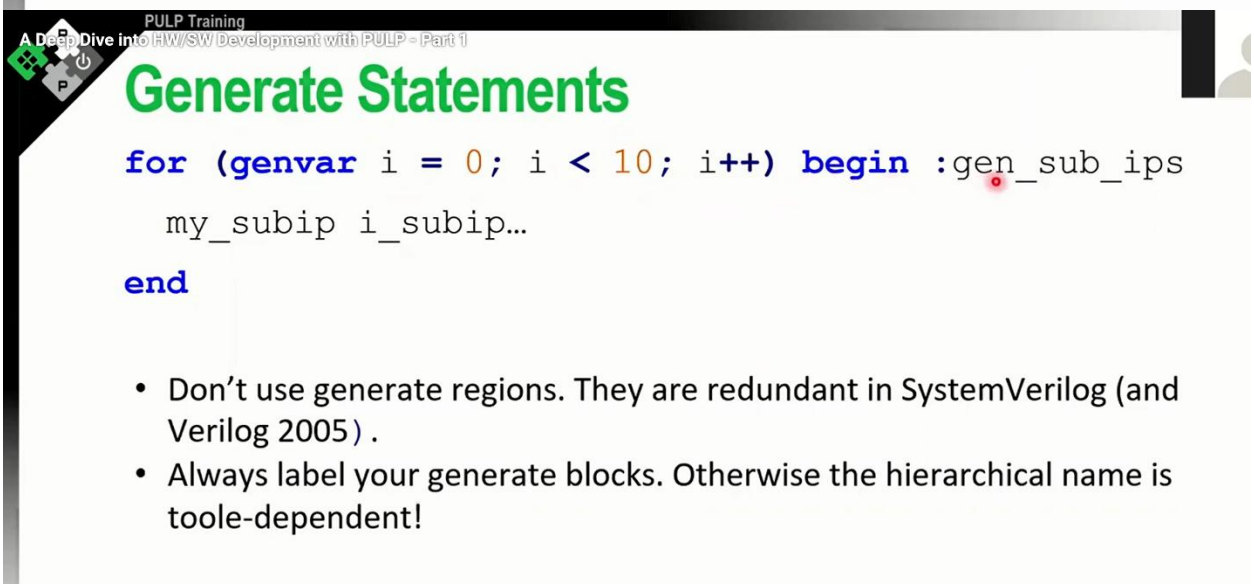
❌ DON’T

- Use `macros.svh`
- Use generic macro names
- Assume macros are scoped
- Rely on tool warnings

7 One-line takeaway

`include` files are global, unscoped, textual copy-paste — so every name inside them must be globally unique.





1️⃣ What a `generate` block actually does (mental model)

A `generate` block is **compile-time hardware replication**.

```
for (genvar i = 0; i < 10; i++) begin
  my_subip i_subip (...);
end
```

This does **not** create a loop in hardware.
It creates **10 separate instances** at elaboration time:

```
my_ip
 ├─ instance 0 of my_subip
 ├─ instance 1 of my_subip
 ├─ ...
 └─ instance 9 of my_subip
```

So hierarchy names **must be deterministic**, or tools and scripts break.

2️⃣ Why labels on generate blocks matter

❌ Unlabeled generate block (BAD)

```
for (genvar i = 0; i < 10; i++) begin
  my_subip i_subip (...);
end
```

Now the tool must invent names.

Depending on the simulator / synthesizer, you may get:

- `genblk1.i_subip`
- `genblk02.i_subip`
- `my_ip.genblk[3].i_subip`
- `GEN_0.i_subip`

👉 **This is tool-dependent and unstable**

Problems this causes:

- Debugging hierarchy paths break
- Waveform scripts stop working
- TCL scripts fail
- Constraints don’t bind

- Cross-tool builds fail (Vivado vs Questa vs Verilator)

This is **poison** for large projects like PULP.

3️⃣ Correct way: always label generate blocks ☑

```
for (genvar i = 0; i < 10; i++) begin : gen_sub_ips
  my_subip i_subip (...);
end
```

Now hierarchy is **guaranteed**:

```
my_ip.gen_sub_ips[0].i_subip
my_ip.gen_sub_ips[1].i_subip
...
my_ip.gen_sub_ips[9].i_subip
```

- ✔ Stable
- ✔ Predictable
- ✔ Tool-independent
- ✔ Script-friendly

This is why the slide says:

“Always label your generate blocks. Otherwise the hierarchical name is tool-dependent!”

4️⃣ Why PULP *especially* cares about this

PULP has:

- many replicated blocks (cores, DMA channels, banks)
- scripting for:
  - synthesis
  - power analysis
  - debug
  - verification
- multiple tools:
  - Verilator
  - Questa
  - Vivado
  - Synopsys DC

If hierarchy names differ → **the entire flow breaks**.

That's why PULP coding rules are strict here.

---

5 "Don't use generate regions" — what does that mean?

Old Verilog style:

```
generate
  for (...) begin
    ...
  end
endgenerate
```

In **SystemVerilog**, this is redundant.

You can just write:

```
for (genvar i = 0; i < N; i++) begin : gen_x
  ...
end
```

Same result, cleaner syntax.

So:

- ❌ `generate ... endgenerate`
- ☑ direct `for (genvar ...) begin : label`

---

6 Quick rule of thumb (memorize this)

If it replicates hardware, it must have a name.

That means:

- generate loops → **label**
- generate if/else → **label**
- parameterized arrays of modules → **label**

---

7 Minimal good example (PULP-style)

```
for (genvar c = 0; c < NR_CORES; c++) begin : gen_cores
  pulp_core core_i (
    .clk (clk),
    .rst (rst)
  );
end
```

Now every core has a stable name:

```
gen_cores[0].core_i
gen_cores[1].core_i
...
```

---

8 Why this is NOT just “style”

This is not aesthetics.

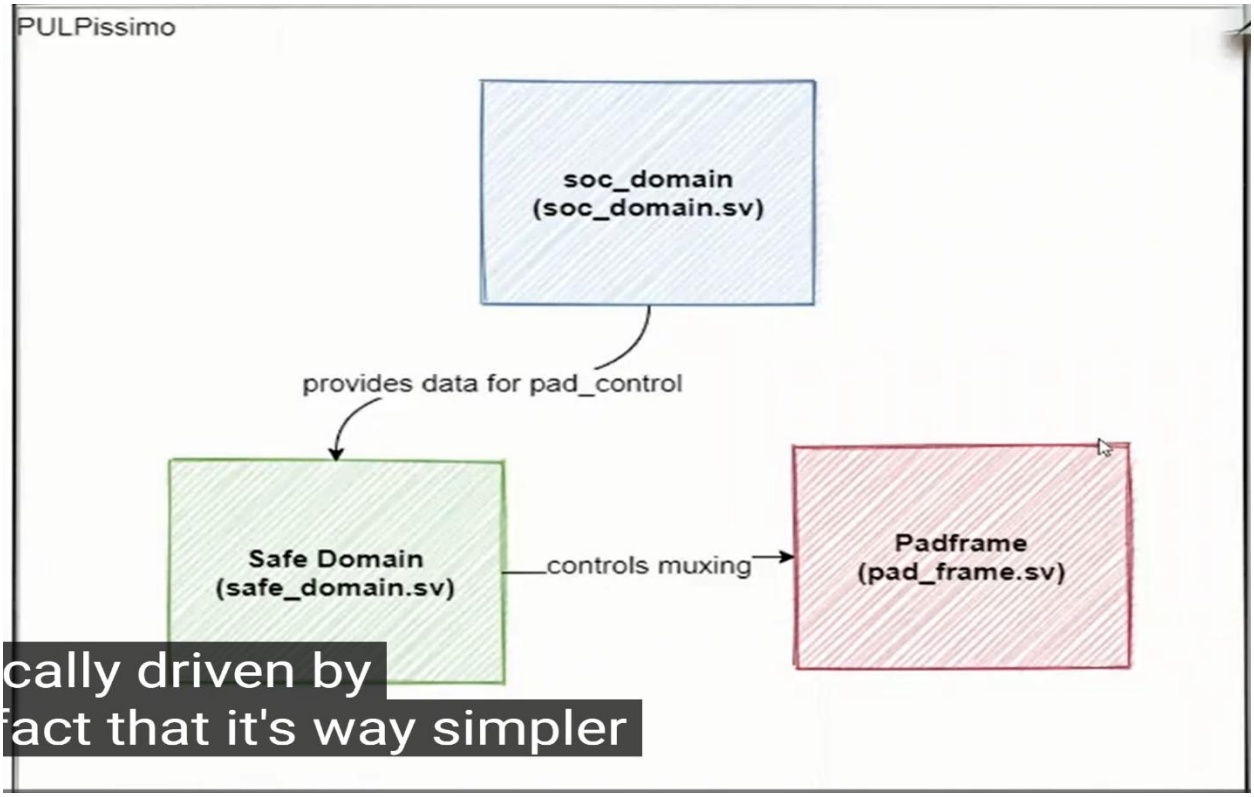
It prevents:

- broken debug
- broken synthesis scripts
- broken constraints
- broken regressions

In SoC-scale projects, **unnamed generate blocks are technical debt**.

Pulpissimo overview notes:

- Logarithmic interconnect (Tightly coupled Data memory interconnect) provide low latency single cycle memory access.
- Micro DMA provides autonomous access for the memory banks from the i/o peripherals
- Hardware processor Engines all have their own master ports to the logarithmic interconnect
- HWPE is programmed by the core by the APB through its configuration interface, the core tells it the start address, what data to process and where to write the results. But data fetch is completely core independent the core is just notified when the HWPE has finished.



Big picture first (what the diagram is saying)

There are **three major blocks** here:

1. **padframe (pad_frame.sv)**
2. **safe_domain (safe_domain.sv)**
3. **soc_domain (soc_domain.sv)**

They are separated mainly for **power management, safety, and clean ownership of IOs**.

---

1 Padframe — *the physical IO boundary*

padframe.sv contains:

- The actual **IO pads** (GPIOs, SPI pins, UART pins, etc.)
- Pad configuration logic:
  - pull-up / pull-down
  - drive strength
  - input/output enable

- muxing between functions

Think of it as:

“Everything that physically touches the outside world.”

⚠️ The padframe **does not decide behavior** by itself — it just *implements* what it is told.

---

2 $afe Domain — *always-on, never dies*

safe_domain.sv contains:

- Logic that must **always remain powered**
- Logic that must be alive **even when the SoC is sleeping or powered down**

Typical contents:

- Pad control registers
- Boot configuration
- Reset logic
- Power control FSMs
- Clock gating / power gating control
- Wake-up logic (GPIO wake, RTC, etc.)

Key idea:

If this logic dies, the chip cannot wake up or boot again.

That’s why it’s called **safe**.

Why pad control lives here

Pads must still:

- Detect wake-up events
- Hold stable values
- Maintain configuration during deep sleep

So:

- **safe_domain controls the padframe**
- It owns the muxes and configuration signals

This matches the arrow:

safe_domain controls muxing → padframe

---

3 SoC Domain — *the "normal" chip logic*

soc_domain.sv contains:

- CPU core
- Interconnect
- Memories
- Peripherals
- DMA
- Accelerators

This is:

"Everything that can be powered down to save energy."

When the chip enters low-power mode:

- **soc_domain can be shut off**
- **safe_domain stays alive**
- **padframe remains electrically stable**

---

4 Why soc_domain "provides data" to safe_domain

The arrow:

soc_domain provides data for pad_control

Means:

- Software running on the CPU writes pad configuration registers
- Those registers physically live in the **safe_domain**
- The soc_domain accesses them via a bus

So:

- **Decision logic** → software in soc_domain
- **Storage & enforcement** → hardware in safe_domain

This split is deliberate:

- CPU can die
- Configuration survives

---

5 Why not put everything in one domain?

Because power gating would be impossible or unsafe.

If pad control were in soc_domain:

- Powering down the SoC would:
  - Randomize IOs
  - Lose wake-up capability
  - Break external communication

This architecture avoids that.

---

6 How this looks in real life

Example: deep sleep

1. CPU configures pads (via soc_domain → safe_domain)
2. CPU goes to sleep
3. soc_domain is powered off
4. safe_domain:
   - Keeps pad config
   - Watches wake-up sources
5. GPIO interrupt fires
6. safe_domain wakes soc_domain
7. CPU resumes execution

Without this split → ❌ impossible.

---

7 Why PULPissimo keeps this *simple*

The tutor hints:

“It’s way simpler”

PULPissimo is:

- A **reference SoC**
- Educational
- Minimal

So:

- Only **one always-on domain**
- Clean separation
- No overly complex power islands

Bigger chips may have:

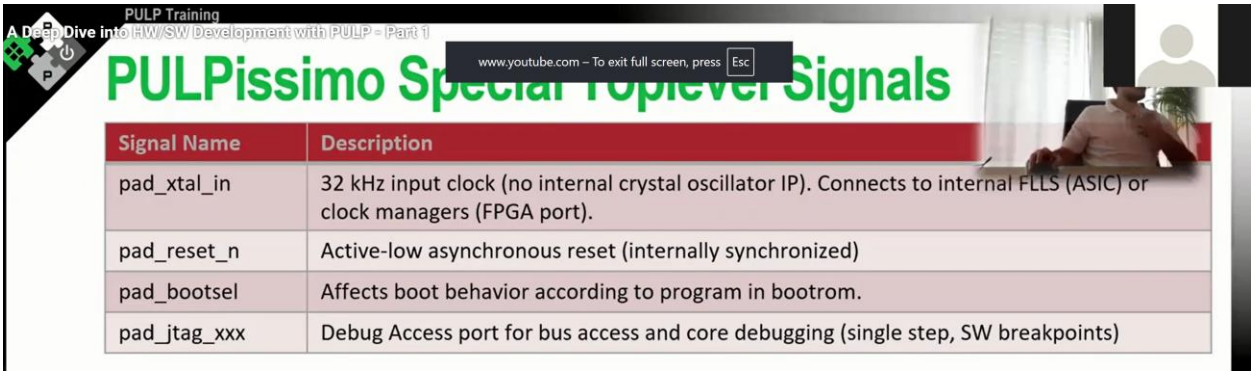
- Multiple safe domains
- Multiple voltage islands
- Retention RAMs
- Partial pad ownership

But PULPissimo keeps it readable.

One-sentence summary (exam-ready)

The padframe contains the physical IOs, the safe_domain contains always-on logic that controls pad configuration and wake-up, and the soc_domain contains the rest of the SoC that can be safely powered down to save energy.

Pulpissimo high level signals:



In the **Pulpissimo** architecture, the top level is divided into three main modules—the **Padframe**, the **Safe domain**, and the **SoC domain**—to facilitate power management and voltage scaling.

Within this structure, several special signals and the `test_mode` signal play critical roles in the chip's operation and testing.

Special Top-Level Signals

The primary top-level signals are essential for clocking, resetting, and determining the initial state of the processor:

- **`pad_xtal_in`**: This is a **32 kHz input clock** that serves as the reference for the internal **Frequency Locked Loops (FLLs)** or clock managers.
- **`pad_reset_n`**: This is an **active-low asynchronous reset** line. It is internally synchronised within the **Safe domain** to the reference clock and further synchronised within the **SoC domain** for faster clock domains.
- **`boot_select`**: This pad determines the boot behaviour of the microcontroller. It functions as a regular GPIO that is read by the software in the **boot ROM** immediately after reset.
    - If **`boot_select` is 0**, the system typically boots from external flash memory (SPI or Hyperflash).
    - If **`boot_select` is 1**, the system enters a **JTAG boot mode**, where the core remains in a busy loop waiting for the debug module to preload memory and take control.
- **JTAG Debug Port**: This port allows for bus access to the entire system and facilitates advanced debugging features for the RISC-V core, such as **single-stepping** and **software breakpoints**. While essential for FPGA and ASIC hardware, it is often bypassed in RTL simulations because the legacy debug module is faster for preloading memory.

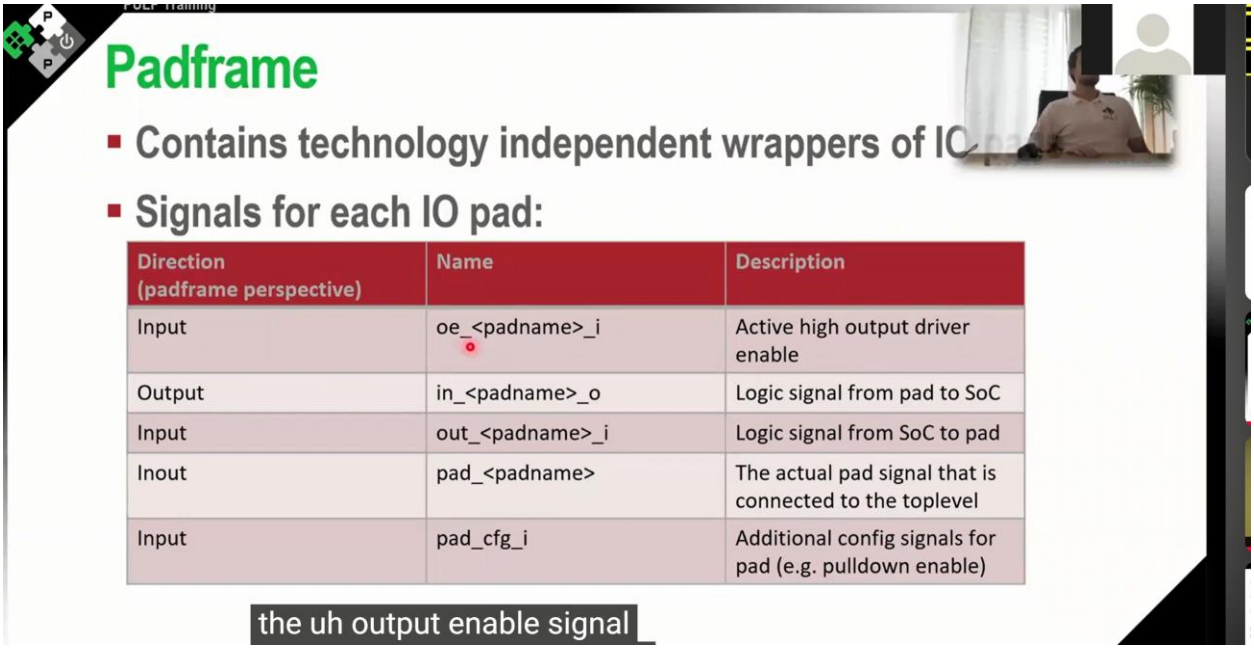
The `test_mode` Signal

The **`test_mode`** signal (sometimes referred to as **`test_enable`**) is a specialized signal propagated throughout the entire hierarchy of Pulpissimo and the SoC.

- **Role in DFT**: Its primary purpose is **Design for Test (DFT)** and scan testing.
- **Interaction with Logic**: The signal is connected to **clock multiplexers** and **manual clock gates**. When `test_mode` is asserted, it forces these gates to stay open, ensuring a continuous clock is applied to the modules. This bypass is necessary because standard scan testing cannot be performed if the clock is gated or disabled during the test procedure.

Interaction Between Domains

These signals interact across the boundaries of the Padframe, Safe, and SoC domains to ensure stable operation. For example, while the **`pad_reset`** enters at the top level, the **Safe domain** contains the reset generator that synchronises it for "always-on" logic. Simultaneously, the **SoC domain** contains the **APB SoC Control** registers, which software uses to configure the pads' driving strength or multiplexing roles, though the actual physical multiplexing logic resides in the **Safe domain**.



In the PULPissimo architecture, the **Padframe** acts as the technology-independent physical interface of the chip, wrapping the specific I/O cells used for either ASIC foundry tape-outs or FPGA implementations. A critical component of these pads is the **pad configuration signal (`pad_cfg_i`)**, which allows for fine-tuned control of the electrical characteristics of each pin.

The Role of Pad Configuration Signals

These signals are provided as a **multi-bit array** (typically 5 or 6 bits per pad) to the technology-dependent wrappers. They are used to manage features that vary significantly between different hardware technologies, such as:

- **Driving strength** (to control how much current the pad provides).
- **Schmitt triggers** (for noise reduction on inputs).
- **Slew rate** control.
- **Pull-up or pull-down** resistor settings (though the first bit is often reserved for this).

Software Programmability and Non-Hardcoded Roles

The sources emphasise that the functional role of each bit within the pad configuration array is **not hardcoded** into the RTL. This flexibility is necessary because different chip manufacturers or FPGA vendors require different control signals for their specific pad cells.

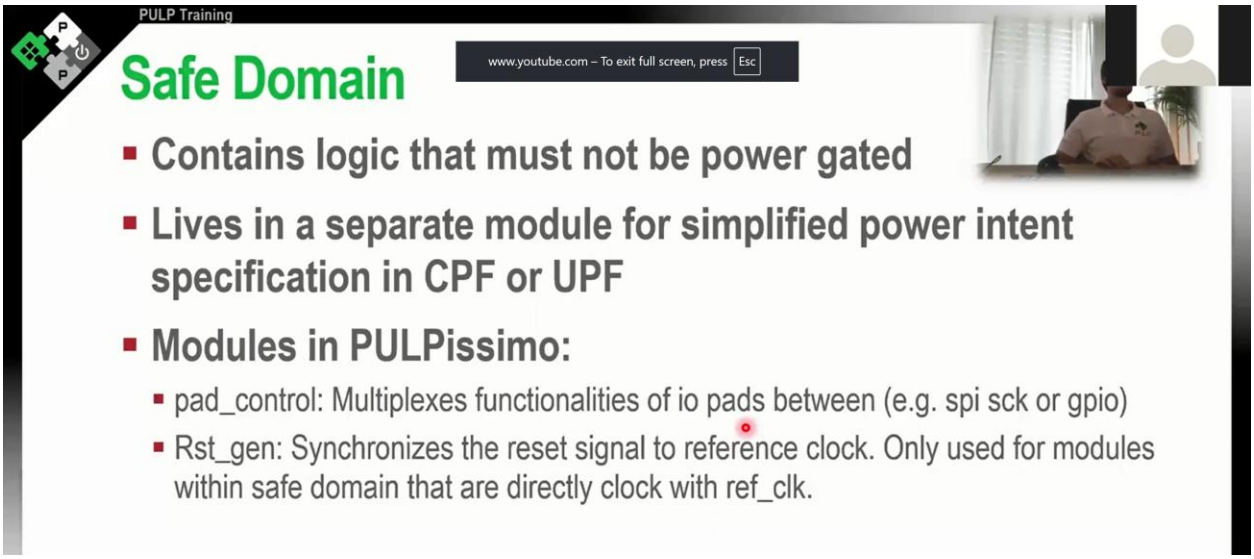
Instead of being fixed in the hardware, these signals are **attached to configuration registers** located within the **APB SoC Control** module, which resides in the SoC domain. Consequently, it is the responsibility of the **software developer** to program the correct bit-pattern into these registers to achieve the desired hardware behaviour for a specific target technology.

Interaction Between Domains

The interaction between the software-driven registers and the physical pads involves several layers of the PULPissimo hierarchy:

- **Storage (SoC Domain):** The configuration values are stored in the **APB SoC Control register file**.
- **Propagation (Safe Domain):** These signals pass through the **Safe domain**, which remains active even when the rest of the SoC is power-gated, ensuring that the pad settings (like pull-up/pull-down status) are maintained during low-power states.
- **Execution (Padframe):** The signals finally arrive at the **Padframe**, where they are applied to the technology-specific I/O buffers to modify their physical operation.

This design ensures that the hardware remains **highly parametric and adaptable**, as the same RTL can be used for various technologies simply by updating the software drivers that manage the pad configuration registers.



In the Pulpissimo architecture, the **Safe domain** and the **Pad control module** play critical roles in managing the chip's power states and its physical interface to the outside world.

The Safe Domain

The **Safe domain** is one of the three primary top-level modules, alongside the Padframe and the SoC domain. Its primary role is to house logic that **must never be power-gated**, ensuring these components remain active regardless of the power state of the rest of the chip. This segmentation is driven by **power intent**, allowing designers to implement different voltage levels or turn off other parts of the system to save energy.

Key components within the Safe domain include:

- **Reset Generator:** This logic synchronises the asynchronous `pad_reset` signal to the 32 kHz reference clock for use by modules within the Safe domain.
- **Pad Control Module:** This is the primary interface between internal peripherals and the physical pads.
- **Always-on Logic:** If the system includes a **power management unit** or **wake-up circuits** (such as those used for sensor data processing), they are typically placed here so they are not affected by power-gating.

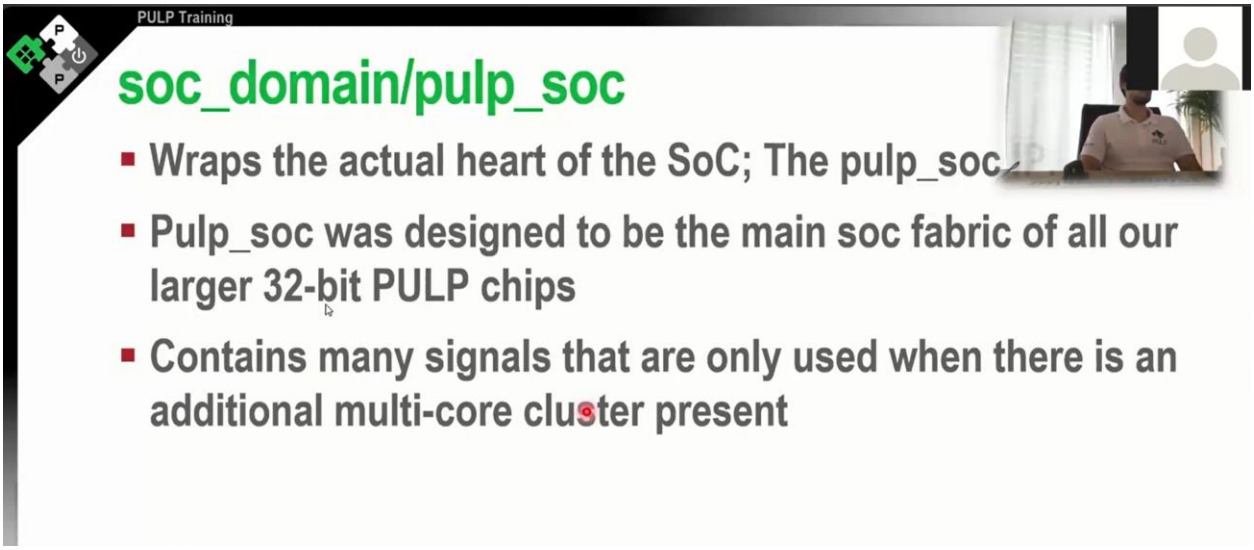
The Pad Control Module

The **Pad control module** functions as a massive **combinatorial multiplexer** for the I/O pads. Its necessity arises from the fact that the number of internal peripheral signals (from IPs like SPI, I2C, and UART) almost always exceeds the number of physical pads available on the chip.

Characteristics and interactions of the Pad Control include:

- **Hardwired Roles:** Every pad has a set of predefined, hardwired roles. For example, a specific pad might be switchable only between acting as an **SPI clock** or **GPIO 0**. You cannot freely route any internal signal to any arbitrary pad.
- **Combinatorial Nature:** The module itself does not store configuration data; it consists purely of multiplexing logic.
- **External Control:** The selection signals that determine which function is active on a pad are supplied by the **APB SoC Control** module, which is located in the **SoC domain**.
- **Software Programmability:** Although the multiplexers are in the Safe domain, they are controlled via software-programmable registers (specifically the `pad_function` registers). By writing to these registers, a developer can change a pad's role, such as switching a UART TX pad to act as a general-purpose GPIO.

This structure ensures that while the high-performance SoC logic can be powered down, the physical interface and its routing configurations remain stable and "safe."



The **SoC domain (Pulp SoC)** serves as the "heart" of the architecture, containing the high-performance logic that constitutes the main functional blocks of the system. It is one of the three primary top-level modules, alongside the **Padframe** and the **Safe domain**, and is specifically designed to be **power-gated** to save energy when high-performance processing is not required.

The SoC Domain (Pulp SoC)

The Pulp SoC provides a shared infrastructure used across different platforms, such as the single-core **PULPissimo** and the multi-core **PULP Open**. Its architecture is built around five major building blocks:

- **Fabric Controller (FC) Subsystem:** The processing core and its immediate accelerators.
- **SoC Peripherals:** Contains I/O peripherals like UART, SPI, and I2C, as well as timers, GPIO controllers, and the **micro DMA** for autonomous memory access.
- **L2 RAM (Multi-bank):** A wrapper for the system's SRAM banks, where address conversion and protocol handling occur.
- **Boot ROM:** The location where the core fetches instructions immediately after a reset.
- **SoC Interconnect:** A logarithmic interconnect (TCDM) that arbitrates between masters (like the FC or DMA) and slaves (L2 RAM or peripherals).

The domain also includes the **SoC Clock and Reset Generator**, which houses the **Frequency Locked Loops (FLLs)** that generate clocks for the SoC, cluster, and peripherals.

The Fabric Controller (FC)

The **Fabric Controller (FC)** is the term used for the primary RISC-V core within the Pulp SoC. In a single-core system like PULPissimo, it functions as the main microcontroller; in multi-core systems, it acts as the **system manager**.

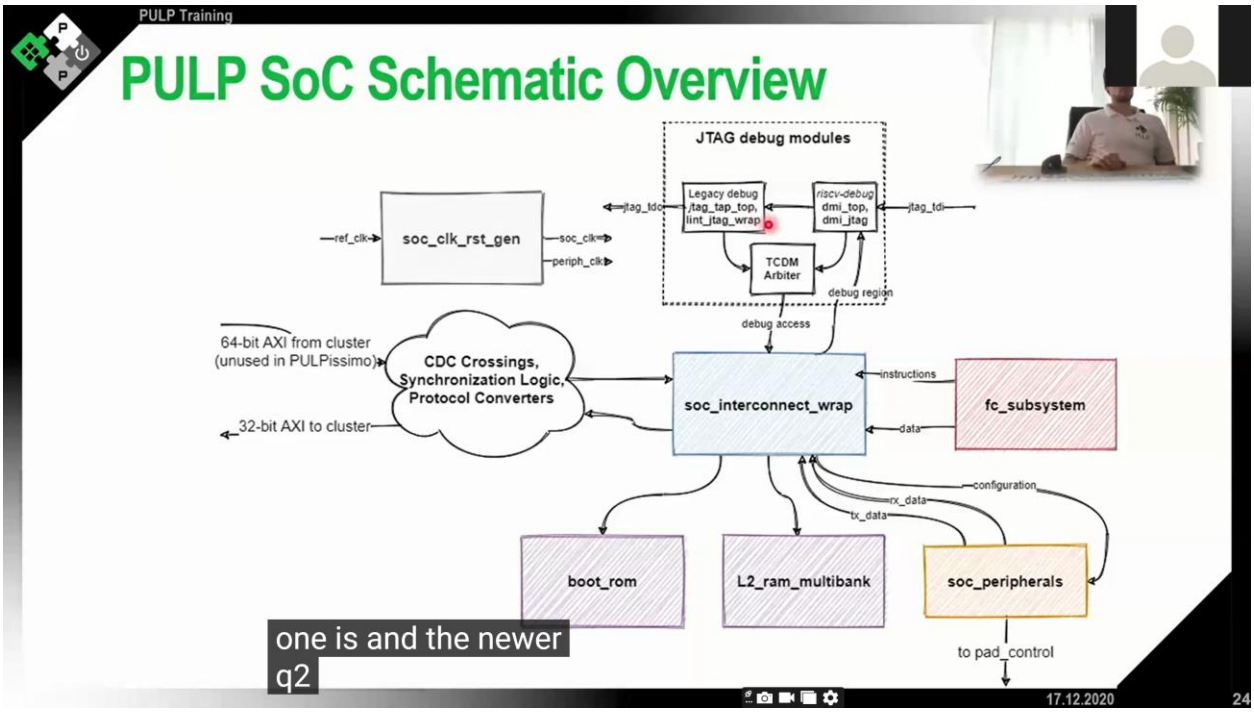
Key roles and features of the FC include:

- **System Orchestration:** In multi-core configurations, the FC is responsible for **configuring I/Os**, **programming the DMA**, and **offloading computational jobs** to the processing cluster.
- **Core Types:** It typically houses 32-bit RISC-V cores such as **risky** (now CV32E40P) or **ibex**.
- **Subsystem Components:** The FC subsystem includes the core, **hardware accelerators** (like the MAC engine), and its own **interrupt controller** with 32 lines.
- **Memory and Peripheral Access:**
    - It connects to the **TCDM bus** (low-latency memory bus) to achieve **single-cycle access** to the L2 memory banks.
    - It communicates with peripherals through the **APB interconnect**, using a memory-mapped interface to route transactions to specific peripheral addresses.
- **Isolation from Accelerators:** While the FC and hardware accelerators live in the same subsystem for power-management reasons, they do **not interact directly**; they

communicate via the SoC interconnect or through the event handler in the peripherals module.

Interaction and Debugging

The FC's operation within the SoC is closely monitored by debug modules. The Pulp SoC contains a **RISC-V compliant debug module** that allows the FC to perform single-stepping and software breakpoints. For faster simulation, a **legacy pulp tab** is also included, which can preload memory via JTAG much more quickly than the standard RISC-V debug specification allows.



PULP SoC – Schematic Overview (Explained)

Think of this diagram as **"how everything talks to everything else"** in a PULP-based SoC.

At the center is the **SoC interconnect**, and everything else either:

- generates traffic (masters), or
- responds to traffic (slaves), or
- helps traffic cross clock / protocol boundaries.

1️⃣ `soc_clk_rst_gen` – Clock & Reset Generator

What it does

- Takes a reference clock (`ref_clk`)
- Generates:
  - `soc_clk` → main SoC clock
  - `periph_clk` → peripherals clock
- Handles resets

Why it matters

- PULP uses **multiple clock domains** (SoC, peripherals, cluster)
- You *cannot* just wire clocks together → needs clean generation & control

---

2️⃣ JTAG Debug Modules (top-right dashed box)

This is **debug access**, not normal program execution.

Two debug paths

1. **Legacy debug**
   - `lint_jtag_wrap`
   - Older PULP debug infrastructure
2. **RISC-V debug**
   - `riscv-debug dmi_top`
   - Standard RISC-V Debug Module Interface (DMI)

TCDM Arbiter

- Arbitrates debug access to **TCDM / memory**
- Ensures debug reads/writes don't collide

➡️ Both debug paths inject **debug traffic** into the SoC interconnect.

3️⃣ CDC Crossings / Synchronization / Protocol Converters (the "cloud")

This is *extremely important*.

CDC = Clock Domain Crossing

Why it exists

- The **cluster**, **SoC**, and **peripherals** may run at:
  - different clocks
  - different voltages
- AXI widths may differ

What it handles

- Clock synchronization
- Safe data transfer
- AXI width conversion

Notes on the labels

- **64-bit AXI from cluster**
  - Used in full PULP SoCs
  - *Unused in PULPissimo* (single-core, simpler)
- **32-bit AXI to cluster**
  - Control/configuration path

➡️ This block prevents metastability and protocol violations.

---

4 `soc_interconnect_wrap` – The Heart of the SoC ❤️‍🩹

This is the **main SoC interconnect**, usually:

- AXI-based
- Multi-master
- Multi-slave
- With arbitration & routing

Who connects here?

Masters (initiators)

- FC subsystem (fetch + load/store)
- Debug modules
- µDMA
- Cluster (in full PULP)

Slaves (targets)

- Boot ROM
- L2 RAM
- Peripherals
- FC instruction & data paths

This block decides:

- **who gets access**
- **to which memory**
- **when**

5️⃣ `fc_subsystem` – Fabric Controller

This is the **main control CPU** (usually single-core RISC-V).

Responsibilities

- Boots the system
- Configures peripherals
- Configures µDMA
- Starts cluster execution (if present)
- Runs OS / runtime code

Two separate paths

- **instruction** path → instruction fetch
- **data** path → loads/stores

➡️ FC is the "brain", not the heavy compute engine.

6️⃣ `boot_rom`

What it is

- Read-only memory
- First code executed after reset

Typical job

- Set stack pointer
- Initialize clocks
- Jump to L2 code
- Possibly enable debug

Think of it as the **BIOS / bootloader** of PULP.

---

7 `l2_ram_multibank`

This is **the main SoC memory**.

Key properties

- Scratchpad (NOT cache)
- Multi-banked
- Shared by:
  - FC
  - µDMA
  - Cluster
  - Peripherals (via DMA)

Why multi-bank?

- Parallel access
- Reduced bank conflicts
- Higher sustained bandwidth

This matches what you learned earlier about **interleaving**.

---

8 `soc_peripherals`

All memory-mapped peripherals live here:

- UART
- SPI
- I2C
- GPIO
- Timers
- Event unit
- µDMA config registers

Connected via:

- `configuration` (APB-style, low speed)
- `tx_data` / `rx_data` (streaming paths)

➡️ Slow control, fast data (classic SoC design).

9️⃣ How data typically flows (example)

Let's follow a **realistic path**:

1. FC configures SPI via `soc_peripherals`
2. FC programs µDMA descriptors
3. µDMA:
   - pulls data from SPI
   - writes into `L2_ram_multibank`
4. FC or cluster processes data
5. Debug can inspect memory via JTAG at any time

All of this traffic is arbitrated by `soc_interconnect_wrap`.

🔑 Key architectural ideas this diagram shows

✔ Separation of concerns

- FC = control
- Cluster = compute
- µDMA = data movement

✔ No caches (intentional)

- Everything is **scratchpad**
- Deterministic timing
- No cache coherence overhead
- Much lower energy

✔ Explicit data movement

- Software knows exactly when and where data moves

- Ideal for embedded + DSP workloads

Relation to PULP vs PULPissimo

PULPissimo

- Single FC core
- No cluster
- Simpler CDC
- Often used for:
  - learning
  - prototyping
  - control-oriented apps

Full PULP

- FC + compute cluster
- 64-bit AXI from cluster
- Much heavier CDC logic
- Designed for high-throughput parallel workloads

➡️ Same architecture philosophy, different scale.

One-sentence mental model

PULP is a scratchpad-based SoC where software explicitly orchestrates data movement, compute, and peripherals through a carefully controlled interconnect.

The **PULPissimo** architecture (and the shared **PULP SoC** infrastructure) is structured into three primary top-level modules: the **Padframe**, the **Safe domain**, and the **SoC domain**. This segmentation is primarily driven by **power intent**, allowing for different voltage levels or the power-gating of high-performance logic while keeping critical interfaces active.

1. The SoC Domain (PULP SoC)

The SoC domain is the "heart" of the system, containing the primary processing and memory resources. It is composed of five major building blocks:

- **Fabric Controller (FC) Subsystem:** This houses a 32-bit RISC-V core (such as **ibex** or **risky/CV32E40P**) and hardware accelerators like a **MAC engine**. These components do not interact directly but communicate via the SoC interconnect or event handlers.
- **SoC Peripherals:** This module contains all I/O peripherals, including **UART, SPI, and I2C**, as well as timers and interrupt controllers. A key component here is the **micro DMA**, which allows these peripherals to autonomously access memory banks without core intervention.
- **L2 RAM (Multi-bank):** This is a wrapper for the system's SRAM banks. It handles **address conversion** (translating 32-bit byte addresses to word addresses) and protocol handling for the memory macros.
- **SoC Interconnect:** A **logarithmic interconnect** (or TCDM bus) that provides single-cycle, low-latency access to memory banks. It also includes an **AXI crossbar** for peripherals that have variable delays, ensuring the system remains compatible with standardized IPs.
- **Boot ROM:** The location where the core fetches its first instructions immediately after reset.

2. The Safe Domain

The Safe domain contains logic that **must never be power-gated**, remaining active regardless of the chip's power state. Its key functions include:

- **Reset Generation:** Synchronising the asynchronous `pad_reset` to the 32 kHz reference clock for "always-on" logic.
- **Pad Control:** This module acts as a massive **combinatorial multiplexer**. Because the chip has fewer physical pads than internal signals, this module routes functional signals (like SPI or GPIO) to specific pads based on software-programmable registers located in the SoC domain.

3. The Padframe

The Padframe consists of **technology-independent wrappers** for the physical I/O pads.

- **Signals:** Each pad includes signals for **Output Enable**, data Input/Output, and a multi-bit **Pad Configuration** array.
- **Configuration:** These configuration bits are not hardcoded; they are attached to registers that software programs to control technology-specific features like **driving strength**, pull-up/pull-down resistors, and Schmidt triggers.

4. System Interconnects and Data Paths

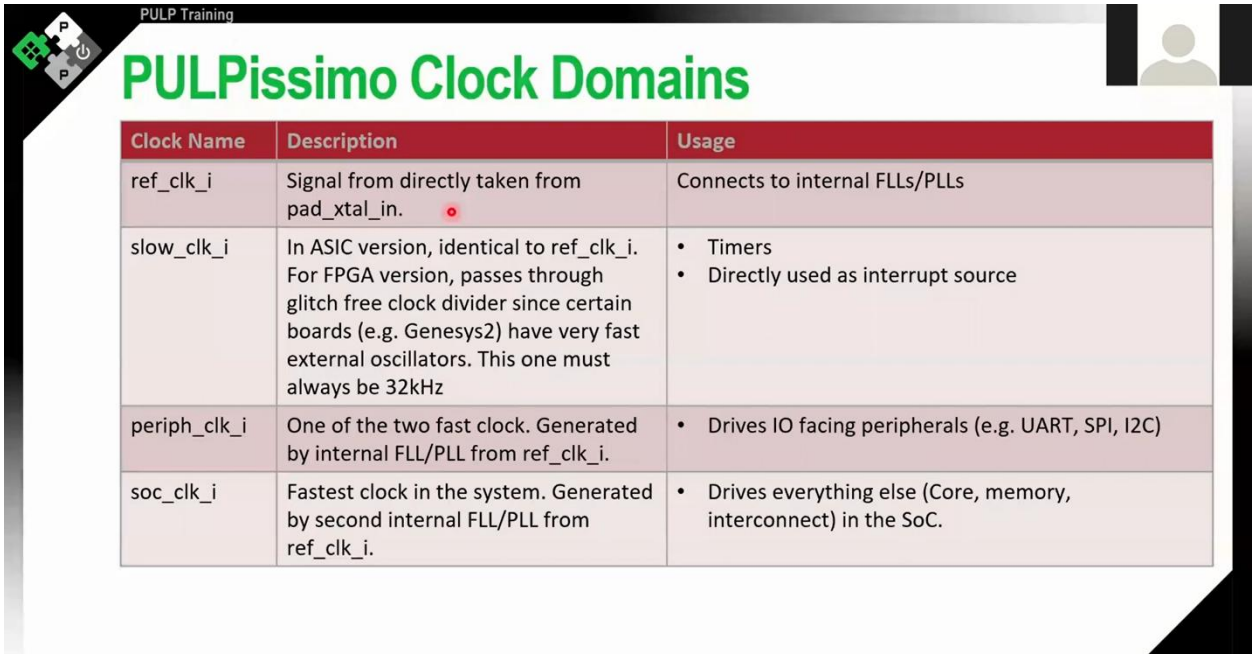
The architecture utilizes two primary bus protocols for internal communication:

- **TCDM Protocol:** A high-performance, single-cycle latency protocol used for memory access by the core, DMA, and debug ports. It supports a throughput of one transaction per

cycle but requires careful management to avoid combinational loops between masters and slaves.
- **APB Interconnect:** Used by the core to communicate with peripheral configuration interfaces through a memory-mapped structure.

5. Debug and Boot Logic

- **Debug Modules:** PULP SoC includes a **RISC-V compliant debug module** for single-stepping and breakpoints, and a **legacy pulp tab**. The legacy module is preferred for RTL simulations because it preloads memory via JTAG much faster than the standard specification.
- **Boot Selection:** The `boot_select` signal is a regular GPIO read by the boot ROM. Depending on its value, the system can boot from external flash (SPI/Hyperflash) or enter a **JTAG boot mode** where the core waits for a debugger to take control.



The PULPissimo and PULP SoC architecture utilises several distinct clock domains to balance performance and power efficiency. These clocks are managed primarily by internal **Frequency Locked Loops (FLLs)** situated within the SoC domain.

1. Clock Domains in PULPissimo

The system is divided into several key clock domains, each serving a specific role in the hierarchy:

- **Reference Clock (`paddock_style_in`):** This is a **32 kHz input clock** provided via the padframe. It acts as the fundamental reference for the internal FLLs or clock managers.
- **Slow Clock:** This is a 32 kHz clock used by **timers** and as an interrupt source. In ASIC implementations, it is identical to the reference clock, while on FPGAs, it is internally divided down from a faster board clock.
- **SoC Clock (`soc_clk`):** This is typically the **fastest clock** in the system, generated by the internal `fll_soc`. It clocks the majority of the logic within the SoC domain, including the Fabric Controller and the interconnect.
- **Peripheral Clock (`periph_clk`):** This domain drives the I/O-facing components of the peripherals. It is generated by a dedicated peripheral FLL.

2. The Cluster Clock

The **Cluster Clock** is an additional domain present only when the SoC is connected to a **multi-core PULP cluster**.

- **Generation:** It is generated by a third FLL, known as the **cluster FLL**.
- **Interaction:** Because the cluster lives in its own clock domain, communication with the SoC domain requires **Clock Domain Crossing (CDC)**.
- **CDC Mechanism:** This interaction occurs through **AXI slices** and FIFOs. These modules exchange pointers using **Grey codes** to ensure stable data transfer between the asynchronous SoC and cluster domains.

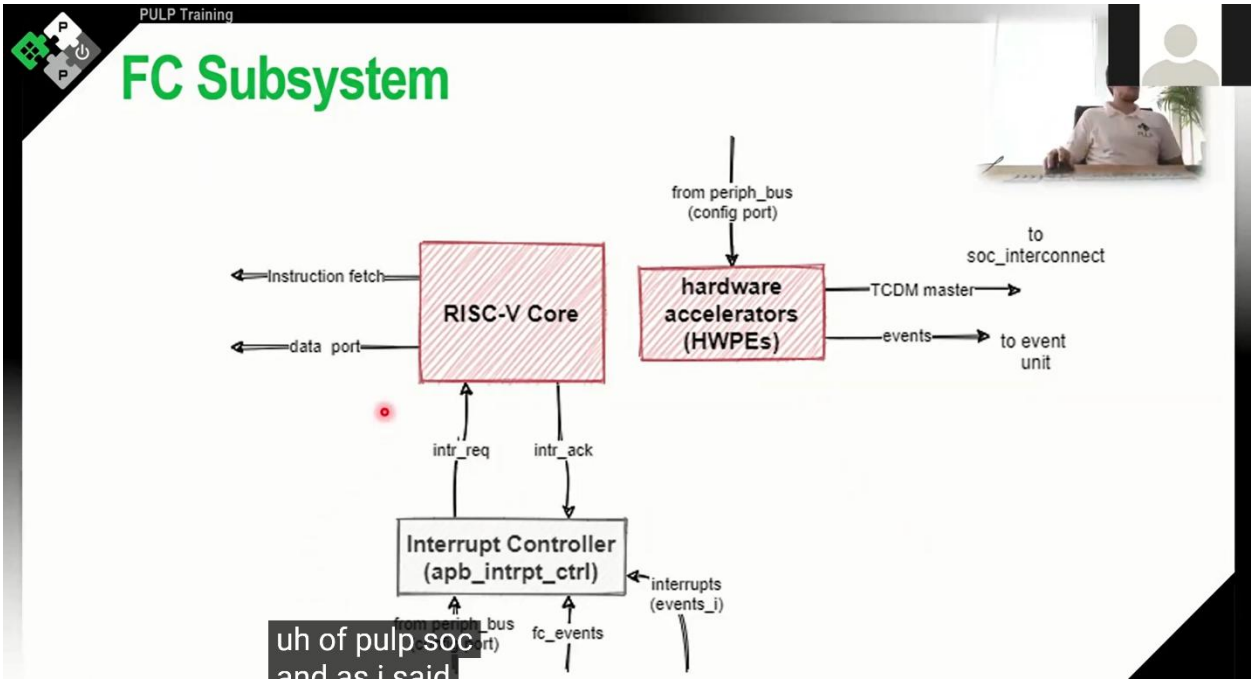
3. Register Configuration for `periph_clk`

The configuration of the peripheral clock is a multi-step process involving both the FLL and the individual peripheral logic:

- **Dual-Clock Peripherals:** Many I/O peripherals, such as the UART, have two clock inputs. The **SoC clock** drives the internal digital logic, such as FIFOs and configuration registers, while the **peripheral clock** drives the actual external signalling (e.g., the baud rate on a TX pad).
- **FLL Management:** The peripheral clock is configured by programming the FLLs located in the **SoC Clock and Reset Generator** module.
- **Internal Divider Calibration:** If the `periph_clk` frequency is changed via software, the **clock divider registers** within the specific peripheral (e.g., the I2C or UART configuration registers) **must be manually updated**.
- **Software Responsibility:** For example, to maintain a specific I2C frequency of 100 kHz after modifying the `periph_clk`, the software developer must reprogram the peripheral's internal divider to match the new master peripheral clock frequency. Changing the SoC clock alone does not affect the toggle speed of the peripheral pads; it only changes the speed of the digital interface logic.

4. Synchronization and CDCs

To manage these disparate domains, PULPissimo implements specific **CDC modules**, typically using **handshaking** or **CDC FIFOs** to prevent meta-stability when signals transition between the SoC, peripheral, and cluster domains.



The **Fabric Controller (FC) subsystem** and **Hardware Processing Engines (HWPE)** are central components of the **SoC domain (Pulp SoC)**. While they reside within the same subsystem for simplified power-management and power-gating intent, they function independently and use specific communication protocols to interact.

The Fabric Controller (FC) Subsystem

The FC subsystem houses the primary 32-bit RISC-V core, such as **Ibex** (formerly zero-riscy) or **CV32E40P** (formerly risky). Its role depends on the system architecture:

- **Single-core (PULPissimo):** It acts as the main microcontroller.
- **Multi-core (PULP Open):** It serves as the system manager, responsible for **configuring I/Os**, **offloading tasks** to the processing cluster, and **programming the DMA**.
- **Components:** In addition to the RISC-V core, the subsystem includes a dedicated **interrupt controller** with 32 lines and local hardware accelerators.

Hardware Processing Engines (HWPE)

HWPE is the term used for hardware accelerators designed with a specific interface that integrates seamlessly into the Pulp SoC structure. Following PULP's **heterogeneous approach**, these engines provide significantly higher energy efficiency for specific tasks than a general-purpose core.

- **Examples:** A common example is a **Multiply-Accumulate (MAC)** engine.
- **Autonomy:** A key feature of HWPEs is that they possess their own **master ports** to the memory interconnect. This allows them to fetch data and write results independently once they have been initialised by the core.

Communication and Interaction

Crucially, the FC core and the HWPEs **do not interact directly**. Instead, their communication is orchestrated through two primary mechanisms:

1. Communication via the SoC Interconnect

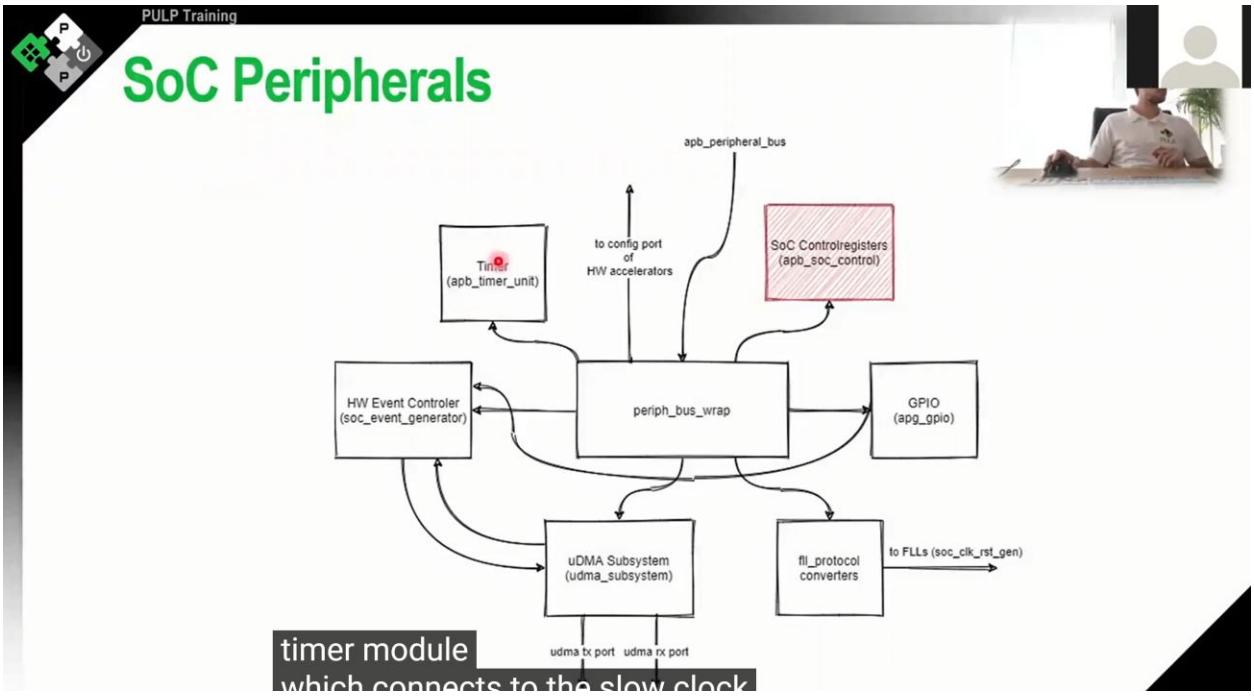
The FC and HWPEs are both masters on the **SoC Interconnect** (also known as the TCDM or tightly coupled data memory bus).

- **Programming:** The FC core programs the HWPE through an **APB configuration interface**. The software tells the accelerator the start address of the data to process and the destination address for the results.
- **Data Transfer:** Once programmed, the HWPE uses its master port to the interconnect to fetch data from the **L2 RAM**. It performs its operation and writes the results back to memory without any intervention from the core.

2. Notification via Event Handlers

Once an HWPE completes its assigned task, it must notify the Fabric Controller.

- **Event Lines:** The HWPE asserts an **event line** or interrupt signal upon completion.
- **Hardware Event Controller:** Because the RISC-V core only has 32 interrupt lines but the SoC generates many more signals, a **hardware event controller** (or event unit) acts as an interrupt extender or multiplexer.
- **Multiplexing:** This controller accepts events from various peripherals and HWPEs, mapping them to the core's available interrupt lines. The core then reads the event controller's registers to determine which specific hardware accelerator or peripheral triggered the interrupt.



The **SoC Peripherals** and the **Fabric Controller (FC)** subsystem reside within the **SoC domain (Pulp SoC)**, the high-performance heart of the chip that is designed to be power-gated,. Because the system's RISC-V core has a limited number of interrupt lines, a specialized **Hardware Event Controller** (also referred to as the **Event Unit**) acts as a crucial intermediary.

SoC Peripherals and HWPE Sources

The **SoC Peripherals** module contains various I/O-facing components, including **UART, SPI, I2C, timers, and GPIO controllers**,,. Additionally, the SoC can include **Hardware Processing Engines (HWPE)**, which are autonomous accelerators (such as a MAC engine),.

Both the standard peripherals and the HWPEs generate signals to notify the core of specific conditions (e.g., data received, a timer expiring, or a hardware task completion),,. In the sources, these signals are often referred to as **events**, though they functionally serve as interrupts.

The Hardware Event Controller (Interrupt Multiplexer)

The primary challenge in the Pulpissimo architecture is that the RISC-V core (such as the risky/CV32E40P) only possesses **32 interrupt lines**,. However, the total number of potential interrupt sources from all peripherals and HWPEs far exceeds this number,.

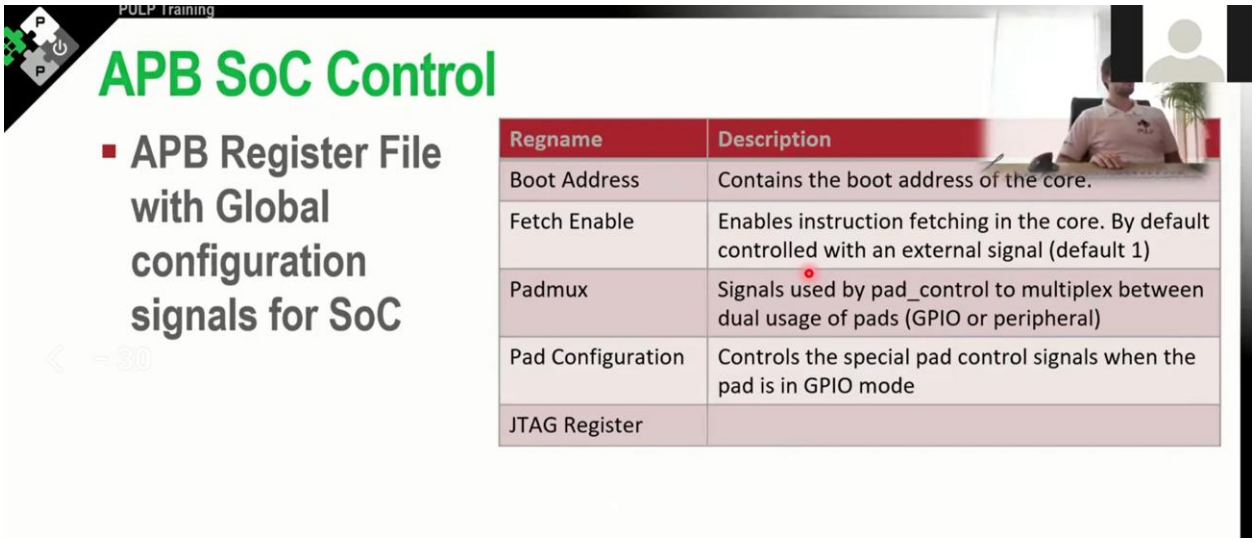
To resolve this, the **Hardware Event Controller** functions as an **interrupt extender or multiplexer**,. Its role and interaction flow are as follows:

- **Reception:** The controller accepts a wide array of interrupt and event signals from all internal peripherals and HWPEs,.

- **Multiplexing:** It multiplexes these numerous incoming signals and maps them onto the core's limited 32 interrupt lines. For instance, while specific lines like line 15 might be assigned to GPIO events, lines **30 and 31** are specifically reserved for the **SoC event generator**,.
- **Core Notification:** When one or more peripheral events occur, the controller asserts a single interrupt line to the core's internal interrupt controller.
- **Software Identification:** Once the core is interrupted, it is the responsibility of the software to communicate with the **Hardware Event Controller's registers**. By reading these registers, the software identifies which specific peripheral or accelerator triggered the event, allowing it to execute the correct driver code,.

Interaction Summary

1. **Peripherals or HWPEs** assert their individual event/interrupt lines,,.
2. The **Hardware Event Controller** receives these signals and groups them,.
3. The controller triggers a specific line (e.g., line 30 or 31) on the **core's interrupt controller**,.
4. The **RISC-V core** enters an interrupt service routine and queries the Event Unit to determine the original source,.



The **APB SoC Control** (also referred to as the **APB Soc_Ctrl**) is a critical IP within the SoC domain, specifically located in the SoC peripherals module. It is essentially a **register file with an APB interface** that stores configuration settings global to the entire SoC.

Software interacts with this module to manage how the chip's internal logic interacts with physical hardware and the boot process. The specific control signals and registers present in this module include:

1. Core Management Signals

- **Boot Address:** This register controls the specific memory address where the core begins fetching instructions after a reset.
- **Fetch Enable:** This signal/register determines whether the core is actively fetching instructions and performing work. By default, it is set to one, meaning the core starts fetching from the boot ROM immediately after reset.
- **Core Status:** This 32-bit register is used by the software runtime to communicate with the simulation environment. When the `main` function terminates, the exit code is written here, allowing the testbench to identify that the simulation is complete.

2. Pad and I/O Control Signals

The APB SoC Control drives the configuration for the other top-level domains (Safe and Padframe):

- **Pad Function (Padmux):** These registers (often four in total) control the large combinatorial multiplexers in the **Safe domain**. They determine the "role" of each pad—for example, whether a pad acts as a UART TX line, an SPI clock, or a GPIO. Each pad typically has two or three hardwired functions that can be selected via these registers.
- **Pad Configuration:** This register holds the multi-bit configuration values for each pad. These signals are propagated to the **Padframe**, where they control technology-dependent physical parameters such as **driving strength**, pull-up/pull-down resistors, and Schmidt triggers.

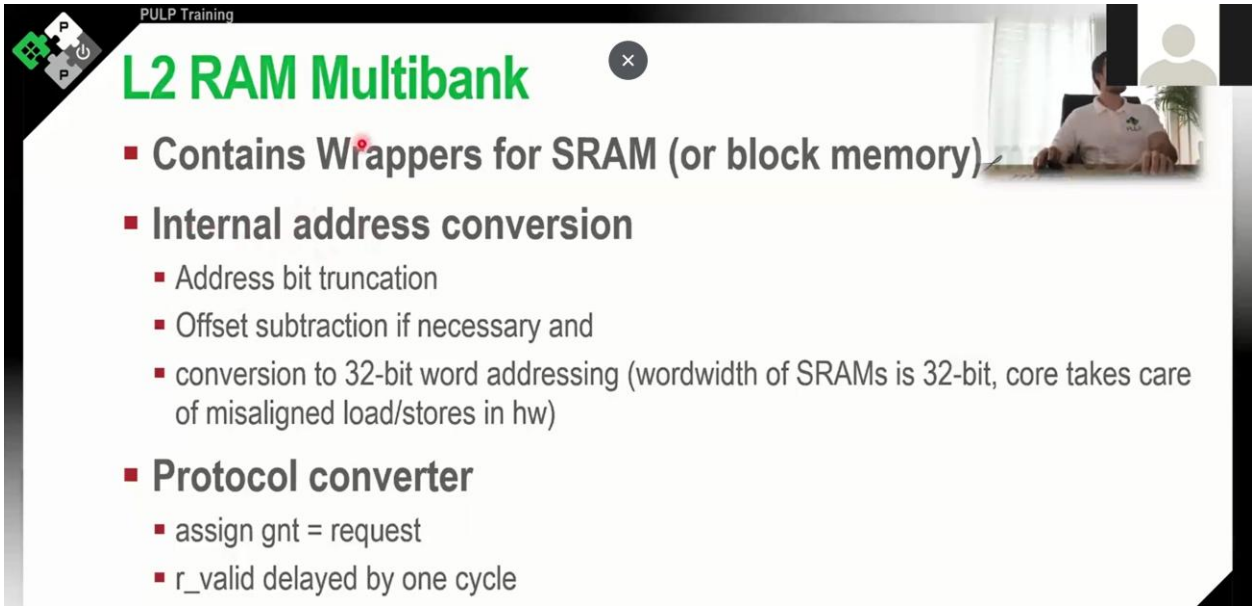
3. Interaction and Debugging Signals

- **JTAG Register:** This is a general-purpose register used for interaction between the JTAG interface and the core. While it does not have a single fixed hardware meaning, it is used by software environments to exchange data during debug sessions.

How These Signals Interact Across Domains

The APB SoC Control acts as the central "brain" for chip configuration. Although the register file itself resides in the **SoC domain**, its signals are distributed as follows:

- **To the Safe Domain:** The **Pad Function** select bits are sent to the Pad Control module to set the multiplexing logic. This ensures that even if the SoC domain is power-gated, the I/O routing remains stable if the Safe domain stays powered.
- **To the Padframe:** The **Pad Configuration** bits are passed through to the technology-specific I/O wrappers in the Padframe to define their electrical characteristics.
- **To the Core:** The **Boot Address** and **Fetch Enable** signals directly influence the Fabric Controller's initial operation during the boot sequence.



In the **PULPissimo** and **PULP SoC** architecture, the **L2 RAM multibank** acts as the primary memory resource within the SoC domain. It is structured as a **wrapper for all SRAM banks** in the system and is responsible for instantiating these banks and managing their interfaces.

1. Key Roles of the L2 RAM Wrapper

The wrapper handles several critical translation and protocol functions to ensure that the memory is compatible with the rest of the SoC:

- **Address Conversion:** While the SoC operates with **32-bit byte addresses**, the individual SRAM macros use a different number of address lines depending on their size. The wrapper converts these 32-bit addresses into the specific format required by the banks.
- **Word Alignment:** In "vanilla" PULPissimo, memory uses **32-bit word addressing** rather than 8-bit addressing. To accommodate this, the wrapper **cuts off the two least significant bits** of the incoming address before applying it to the SRAMs.
- **Offset Handling:** Because some memory regions do not start at "natural" boundaries in the system's address space, the wrapper **subtracts a specific offset** from the incoming address so that it correctly maps to the start of the SRAM macro.
- **Protocol Conversion:** The wrapper manages the handshaking and protocol requirements for the SRAMs. Since standard SRAMs do not inherently use a handshaking protocol, the wrapper enables the correct bank and manages the single-cycle delay for read responses.

2. Memory Banking and Interleaving

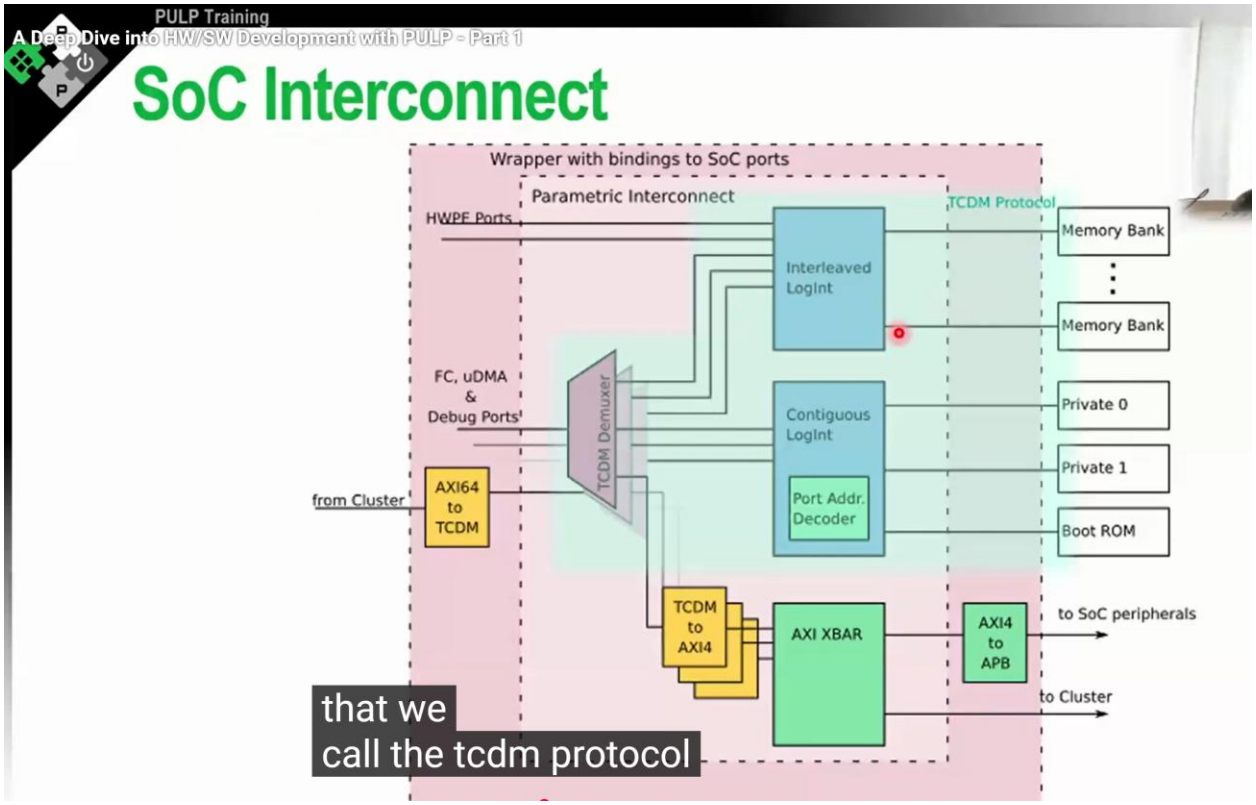
The L2 RAM is divided into different regions to optimize performance and reduce **bank conflicts** when multiple masters (like the Fabric Controller, micro DMA, and hardware accelerators) access memory simultaneously:

- **Interleaved Region:** This is the bulk of the system memory. It uses a **word-interleaving** scheme where sequential 32-bit word addresses are mapped to different physical memory banks (e.g., address 0 to bank 0, address 4 to bank 1, etc.). This design is intended to **reduce bank conflicts** when high-performance masters like hardware accelerators access large data arrays.
- **Contiguous (Private) Banks:** PULPissimo typically includes two "private" banks (Bank 0 and Bank 1). Unlike the interleaved region, these are **sequentially mapped**.
    - **Private Bank 0:** Typically used for the **system stack**.
    - **Private Bank 1:** Typically used for the core's **instructions**.
    - **Purpose:** By placing instructions and the stack in these dedicated banks via the linker script, the Fabric Controller can avoid bank conflicts with the DMA or accelerators that are primarily operating in the interleaved data region.

3. Interaction and Performance

The L2 RAM is connected to the masters via a **logarithmic interconnect** (TCDM bus), which provides **single-cycle, low-latency access**. The TCDM protocol is specifically designed for high-throughput memory access, supporting one transaction per cycle.

During system development, the L2 RAM layout is highly parametric. Developers can modify the **banking factor**, change the total amount of SRAM, or add additional private banks to suit specific application requirements.



The **SoC interconnect** (often referred to as the **TCDM bus** or **logarithmic interconnect**) is the central arbitration system within the PULPissimo architecture, providing **single-cycle, low-latency access** to the system's memory banks. It facilitates communication between various internal masters and the memory-mapped slaves of the system.

Core Masters and Slaves

The interconnect manages transactions from several high-performance masters, including:

- **Fabric Controller (FC):** Uses dedicated instruction and data ports.
- **Micro DMA:** Employs independent TX and RX channels to move data between peripherals and memory without core intervention.
- **Debug Modules:** Both the RISC-V compliant debug module and the legacy "pulp tab" access the system bus through a shared arbiter.
- **Hardware Processing Engines (HWPE):** These accelerators have their own master ports to fetch data independently.

The primary slaves connected to the interconnect are the **L2 RAM**, the **Boot ROM**, and the **SoC peripherals** (accessed via a protocol converter).

Memory Regions and Address Routing

Transactions are routed to different domains based on **address rules** defined in configuration header files. The interconnect segments memory into three primary regions:

- **Interleaved Region:** Sequential 32-bit word addresses are mapped across different physical banks. This **word-interleaving** reduces bank conflicts when multiple masters, such as HWPEs and the core, access large data arrays simultaneously.
- **Contiguous (Private) Region:** Address space is mapped sequentially to specific banks, such as **Private Bank 0** (typically for the stack) and **Private Bank 1** (for instructions). This ensures the core can access critical data without interference from other masters.
- **AXI Crossbar:** Any address not specifically mapped to the TCDM regions is automatically routed to the **AXI crossbar**. This component handles peripherals with **variable or dynamic delays**, which are not suited for the rigid timing of the TCDM protocol.

Communication Protocols

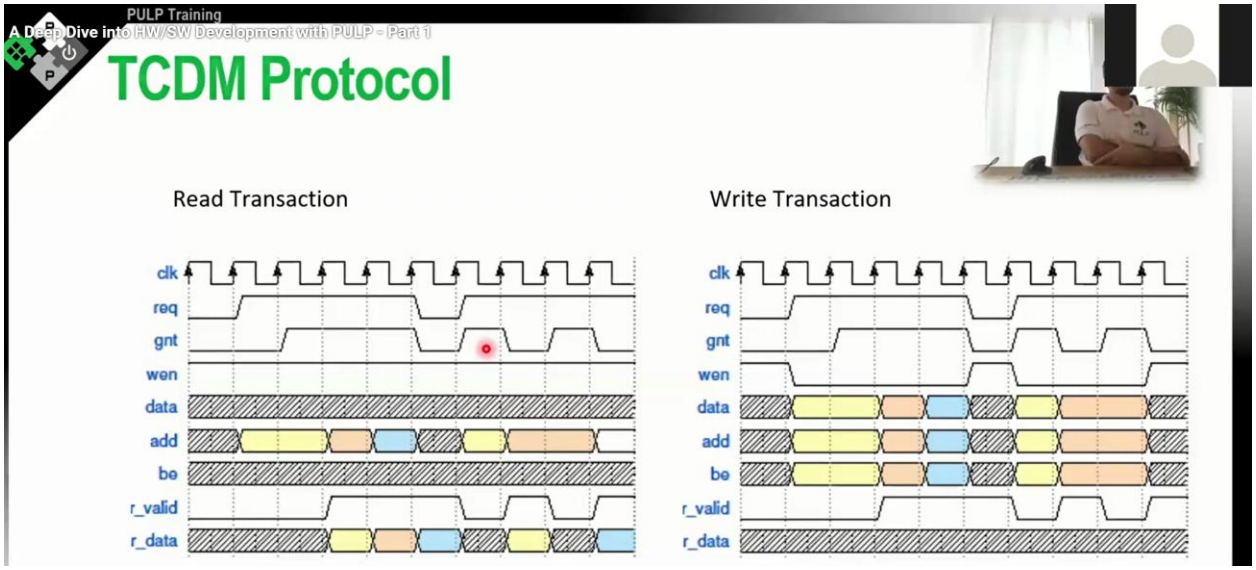
The interconnect primarily uses the **TCDM protocol**, a high-speed handshake-based system:

- **Handshake:** A master asserts a `request` while the slave asserts a `grant` to accept a transaction within the same cycle.
- **Throughput:** It supports a throughput of one transaction per cycle, with the `r_valid` signal typically asserted exactly one cycle after the grant for read operations.
- **Design Risks:** Because the protocol allows `grant` to depend combinationally on `request`, it is highly susceptible to **combinational loops**. Designers must ensure the

`request` line does not depend on the `grant` or `r_valid` signals within the same cycle to avoid synthesis failures.

Parametric Nature

The modern PULPissimo SoC interconnect is highly **parametric**. It consists of a wrapper that instantiates a core interconnect with a configurable number of master ports, interleaved slaves, and AXI slave ports. This allows developers to easily scale the system by re-parameterising the module to accommodate new hardware accelerators or additional memory banks.



The **Tightly Coupled Data Memory (TCDM)** protocol is a high-performance, **single-cycle latency** protocol used in the PULPissimo and PULP SoC architectures to facilitate rapid communication between masters (such as the core, DMA, or hardware accelerators) and memory slaves.

Key Signals of the TCDM Protocol

The protocol relies on a specific set of signals to manage memory transactions:

- **`request`**: Asserted by the master to initiate a transaction.
- **`address`**: The target memory address provided by the master.
- **`WEN` (Write Enable Negated)**: An **active-low** signal; the master pulls this to zero to perform a write and keeps it high for a read.
- **`be` (Byte Enable)**: Used to write to specific bytes within a word (though not all IPs support this).
- **`grant`**: Asserted by the slave to signal that it has accepted the transaction.
- **`r_valid`**: Asserted by the slave to indicate that the data on the read bus is valid.
- **`r_data`**: The data bus for read responses.

Read and Write Transactions

A transaction is considered valid and takes place during the cycle in which both the **`request`** and **`grant`** signals are asserted simultaneously.

- **Read Transaction:** To perform a read, the master asserts `request` while keeping `WEN` high. If the slave accepts the transaction immediately, it asserts `grant` in the same cycle. The read data is then typically delivered via `r_data` exactly **one cycle after** the `grant` was asserted, accompanied by the `r_valid` signal.
- **Write Transaction:** To perform a write, the master asserts `request` and pulls `WEN` to logic low. Because the data is provided alongside the request, the transaction is effectively completed from the master's perspective as soon as the slave asserts `grant`.
- **Throughput:** The protocol supports a throughput of **one transaction per cycle**. This allows for "back-to-back" transactions where a new request is granted every cycle.

Critical Protocol Rules and Timing

- **Combinational Grant:** To achieve single-cycle latency, the slave is permitted to assert the `grant` signal **combinationally** based on the incoming `request` in the same clock cycle.
- **Avoiding Combinational Loops:** A fundamental rule of the TCDM protocol is that the **`request` line must not combinationally depend on the `grant` line**. If a master's request logic depends on the slave's grant or valid signals within the same cycle, it will create a combinational loop that prevents the design from being synthesised.
- **Back Pressure:** The slave can apply back pressure by delaying the assertion of the `grant` signal (e.g., if there is a bank conflict). However, some IPs in the PULP ecosystem are rigid regarding the read response and only support `r_valid` being asserted exactly one cycle after the `grant`.

AXI Cross bar

In the PULPissimo architecture, the **SoC Interconnect** is managed by a top-level wrapper, `soc_interconnect_wrap.sv`, which integrates the **AXI crossbar** to handle standardized and variable-delay communication. The following details outline the configuration, role, and port mapping of these components.

The AXI Crossbar

The AXI crossbar is a relatively new addition to the interconnect, designed to manage interactions that are not suitable for the rigid, single-cycle **TCDM protocol**.

- **Role:** It handles peripherals or modules with **variable or dynamic delays**, where it is not possible to decide within a single cycle if a slave can accept a transaction.
- **Standardisation:** Using AXI makes the SoC compatible with standardized IP blocks outside the PULP ecosystem.

- **Output Ports:** In a standard PULPissimo configuration, the AXI crossbar features **two output ports**:
    - **Port 0:** Connects to the **pulp cluster** via a 32-bit AXI plug (unused if no cluster is present).
    - **Port 1:** Connects to the **SoC peripherals** via a protocol converter that translates AXI4 to APB.
- **Catch-all Routing:** The AXI crossbar acts as a **default domain**; any address that does not explicitly match the rules for the interleaved or contiguous TCDM memory regions is automatically routed here.

Configuring `soc_interconnect_wrap.sv`

The `soc_interconnect_wrap.sv` file serves as a bridge between the specific signals of the SoC and a highly **parametric core interconnect**.

- **Parametric Design:** The internal core interconnect is generic and does not "know" about specific ports like "Instruction Fetch". It is configured with a variable number of master ports, interleaved slave ports, contiguous slave ports, and AXI slave ports.
- **The Wrapper's Role:** It is the job of the wrapper to instantiate these ports and attach the correct functional interface to each one.
- **Address Decoding:** The wrapper also specifies the **address rules**. It uses start and end addresses (often defined in an external header like `soc_mem.h`) to configure the internal address decoder IP.

The Ports Mapping

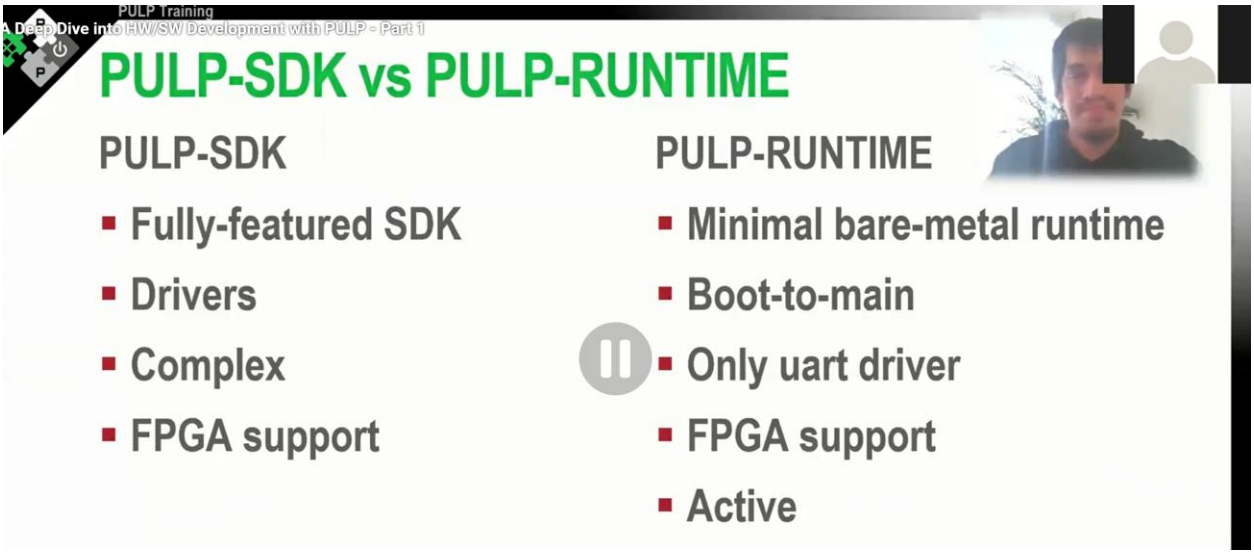
The interconnect manages various masters and slaves through the following mappings:

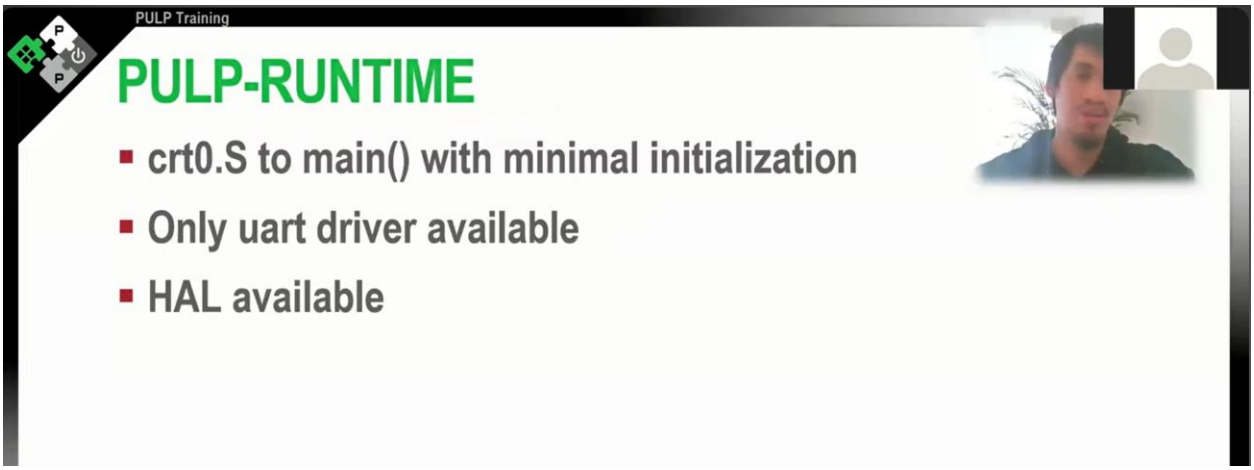
Master-Side Ports (Initiators)

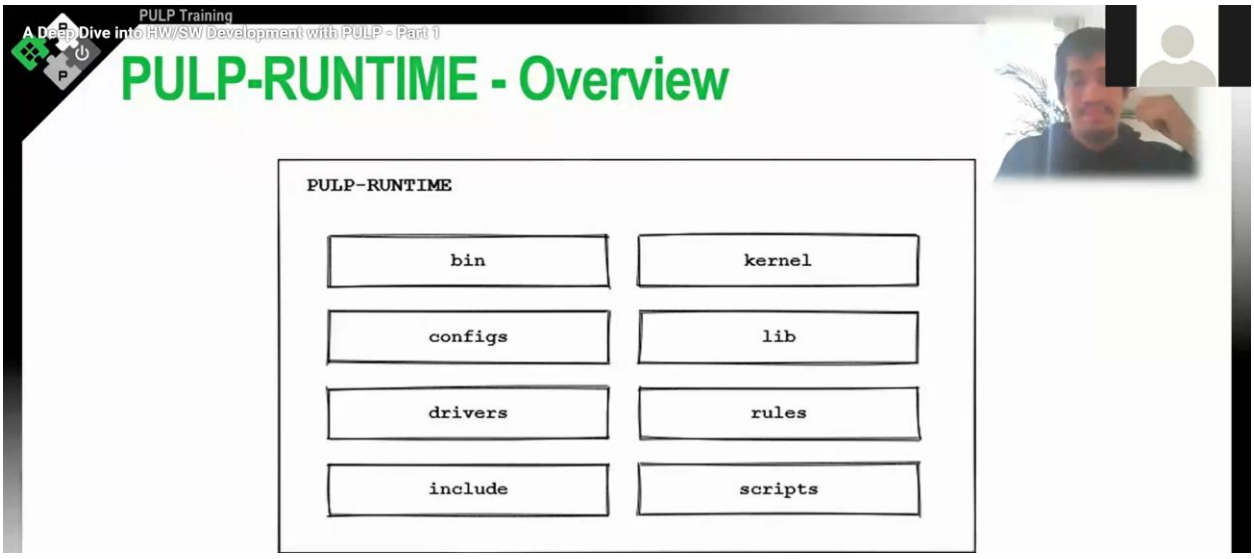
1. **Fabric Controller (FC) Data Port:** The primary data interface for the RISC-V core.
2. **FC Instruction Port:** Dedicated port for the core's instruction fetches.
3. **Micro DMA TX Channel:** Used by the DMA to write data to memory.
4. **Micro DMA RX Channel:** Used by the DMA to read data from memory.
5. **TCDM Debug Port:** A shared port that uses a demultiplexer to arbitrate between the **RISC-V debug module** and the **legacy pulp tab**.
6. **Hardware Processing Engines (HWPE):** A configurable number of master ports for autonomous accelerators (e.g., the MAC engine uses four).
7. **Cluster Master Port:** A 64-bit AXI interface used by the cluster to fetch instructions and data from L2 memory (unused in single-core PULPissimo).

Slave-Side Ports (Targets)

1. **Interleaved Region:** Connects to the bulk of the L2 RAM banks, providing single-cycle access using word-interleaving to reduce bank conflicts.
2. **Contiguous Region:** Sequential address mapping for specific resources:
    - **Private Bank 0:** Often used for the system stack.
    - **Private Bank 1:** Often used for core instructions.
    - **Boot ROM:** Read-only access for initial boot instructions.
3. **AXI Crossbar:** As noted above, this port leads to the cluster interface and the APB peripheral subsystem.







The **PULP-runtime** is a low-level software environment designed to transition the system from the boot ROM to the `main` function. It provides a minimal C runtime and is intended for developers who need to interact directly with hardware registers or write lightweight drivers without the overhead of a complex toolchain.

Based on the sources, the following is a breakdown of the function of each folder within the PULP-runtime:

1. Kernel

Despite its name, this is not a full operating system kernel but a historical designation for the core **initialisation logic**. Its primary roles are to:

- Configure the **Frequency Locked Loop (FLL)** to generate a stable system clock.
- Initialise the **interrupt controller**, typically by disabling all interrupts at the start.
- Set up the **C runtime (CRT0)** and the **stack**, which are required for execution to transition to the `main` function.

2. Bin

This folder contains binaries and scripts essential for interacting with hardware and simulations. A critical tool found here is the **`stimutils`** Python script, which converts `.elf` binaries into formats that can be loaded into an FPGA or the RTL testbench. It also houses GDB scripts and tools for specific FPGA boards, such as the Genesis 2.

3. Lib

This directory provides a **minimalist C library** rather than a full `libc`. It is specifically designed to be extremely small to prevent code bloat, offering basic functions like **`printf`** and **`exit`** primarily for debugging purposes.

4. Configs (Configuration)

The configuration folder contains shell scripts that a developer must **source** into their environment before working with the runtime. These scripts define critical environment variables, such as the **target hardware platform** (e.g., PULPissimo) and the paths to the runtime itself, which allows the makefiles to know which specific platform settings to apply.

5. Include

This is the repository for **header files** (`.h`). It includes `pulp.h`, which serves as a **global include file**; by including `pulp.h` in a project, all other necessary headers for the specific target platform are automatically pulled in.

6. Drivers

This folder contains low-level hardware drivers. In its current state, it is kept minimal, primarily providing a **UART driver** to facilitate basic communication and debugging on real hardware or FPGAs.

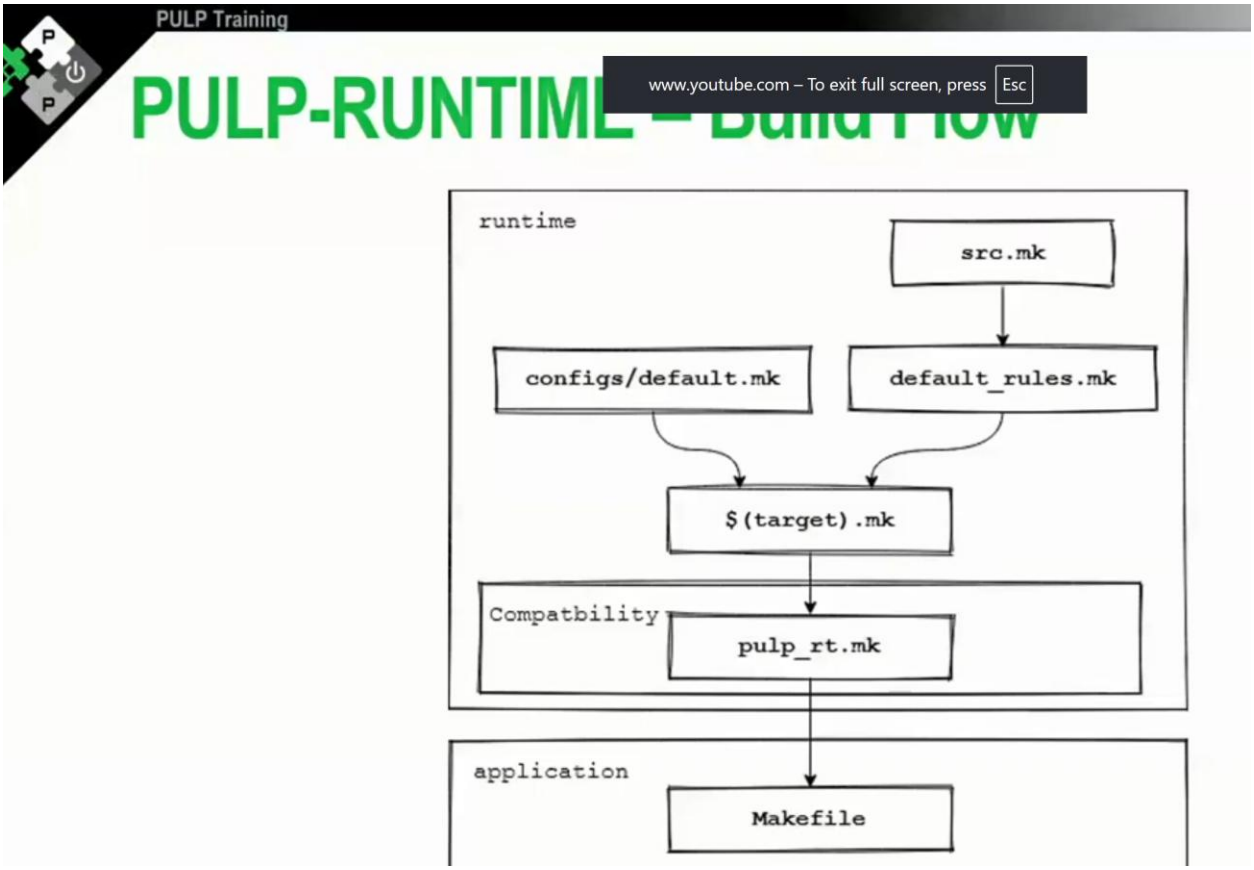
7. Rules

The rules folder contains **makefile fragments** used to manage the build flow. These files are structured in a hierarchical fashion: an application includes a top-level makefile from this folder, which then includes platform-specific rules based on the variables set in the `configs` folder. This system defines all targets for **compiling** the application and **launching simulations**.

8. Scripts

This folder houses various helper scripts to assist with testing and debugging:

- **`plp_run_test`**: A script used in continuous integration (CI) to run tests and generate XML result files.
- **`pulp-trace`**: A crucial debugging tool that takes a raw execution trace (hexadecimal addresses) and the binary file to produce a **symbolised trace**, mapping the hex values to actual function names and instructions in the source code.



The **Pulp-runtime** provides a low-level software environment designed to transition the system from the boot ROM to the `main` function. It features a minimalist build system based on hierarchical makefiles that manage compilation and simulation across different platforms.

1. The Build Flow Infrastructure

The Pulp-runtime build system is structured as a series of **nested makefile fragments**. This design ensures that the same application can target different hardware platforms (like FPGA or RTL simulation) simply by changing environment variables.

- **Application Makefile:** Every application (such as "Hello World") contains a simple makefile that sets basic variables and then **includes `pulp_rt.make`**.
- **Platform Selection:** `pulp_rt.make` uses the `PULP_RT_TARGET` environment variable to include the correct platform-specific makefile (e.g., `pulpissimo.make`).
- **Compilation Flags:** The platform-specific makefile defines the **C compiler flags (`CFLAGS`)**, linker flags, and the specific instruction set architecture (ISA) being targeted.
- **Target Definitions:** Finally, `default_rules.make` is included, which contains the actual commands for the `all` (compile) and `run` (simulate) targets.

2. Flow for Running the "Hello" Example

Running a test example like "Hello World" follows a specific sequence of environment configuration, compilation, and execution.

Step 1: Environment Setup

Before running any commands, you must configure your shell so the makefiles can find the runtime, the compiler, and the simulator.

- **Source Configuration:** Source the platform-specific script (e.g., `source configs/pulpissimo.sh`). This sets `PULP_RT_TARGET` and `PULP_RT_HOME`.
- **Pathing:** Ensure the **RISC-V GCC toolchain** is in your system path or set via the `RISCV_GCC_TOOLCHAIN` variable.
- **Simulator Path:** Source the simulator setup script (e.g., `pulpissimo/setup.vsim`) to set the `VSIM_PATH`, which tells the runtime where the RTL simulation platform is located.

Step 2: Compilation (`make all`)

Navigating to the `hello` directory and running `make all` triggers the following:

- **Binary Generation:** The RISC-V compiler builds the application into an **ELF binary**.
- **Stimulus Conversion:** The build flow invokes the **`stimutils` Python script**. This script converts the ELF binary into a format that the RTL testbench or FPGA can actually load into memory.

Step 3: Simulation (`make run`)

Running `make run` executes the test:

- **Simulator Launch:** The runtime calls the RTL simulator (typically **Questasim/VSIM**).
- **Execution:** The core boots from the ROM, initializes the stack and hardware (like the FLL), and jumps to your `main` function.
- **Termination:** The testbench monitors a specific **core status register**. When your program finishes, the runtime writes the exit code to this register, signaling the testbench to stop the simulation.

3. Post-Run Analysis and Debugging

After the simulation completes, the `build` folder contains several files for analysis:

- **Terminal Output:** The "Hello World" message and the return value (e.g., `0` for success) are printed to the console.
- **Execution Logs:** A file named `trace_core_00_0.log` provides a raw cycle-by-cycle execution trace, including program counters and hexadecimal instructions.
- **Symbolised Traces:** Because raw logs are hard to read, you can run the **`pulp-trace` script**. This tool maps the hexadecimal addresses in the log back to the actual function

names and instructions in your source code, making it much easier to see where a program might have failed.

Build Folder contents that helps in debugging:

To run a test application like "Hello World" and debug its execution in the PULPissimo environment, you must follow a specific sequence of environment setup and compilation steps. Once the simulation is complete, the generated files in the build folder provide essential insights for both software and hardware debugging.

Steps to Run a Test Application

The execution flow relies on configuring the environment variables so that the makefiles can locate the compiler, the runtime, and the RTL simulation platform.

1. **Configure the Toolchain:** Point the environment to your RISC-V GCC toolchain by setting the `RISCV_GCC_TOOLCHAIN` variable or extending your system path.
2. **Source Runtime Configuration:** Source the target-specific configuration script in the runtime folder (e.g., `source configs/pulpissimo.sh`). This sets the `PULP_RT_TARGET` and `PULP_RT_HOME` variables.
3. **Source Simulator Setup:** Source the simulation setup script (e.g., `source setup.vsim` in the `sim` folder) to set the `VSIM_PATH`, allowing the environment to find the compiled RTL platform.
4. **Compile (`make all`):** Running `make all` in the application directory generates an ELF binary and invokes the **`stimutils`** Python script to convert that binary into a stimulus format (like a `.cde` file) that the RTL testbench can load into memory.
5. **Simulate (`make run`):** This command launches the simulator (e.g., QuestaSim). The simulation ends when the software writes an exit code to the **core status register**, which the testbench polls to determine termination.

Build Folder Contents for Debugging

After running a simulation, the `build` folder contains several critical logs and traces:

- **`stdout/` Folder:** This directory contains the standard output captured during simulation. For example, `p310.log` captures the output of the Fabric Controller.
- **`trace_core_00_0.log`:** This is a raw execution trace. It lists every instruction executed by the core, showing the **cycle count, program counter (PC), and the hexadecimal representation** of the instruction.
- **`uart/` Folder:** This captures output directed to the UART peripheral, which is particularly useful when debugging code intended for FPGA or real hardware.

Advanced Debugging Tools

Because raw execution traces consist of hexadecimal addresses that are difficult to interpret manually, the sources recommend two primary methods for deeper analysis:

- **`pulp-trace` Script:** You can run this helper script by providing it with the raw `trace_core_00_0.log` and the application's ELF binary. It produces a **symbolised trace** that maps hexadecimal addresses to actual function names (like `main`) and assembly mnemonics, making the software flow much easier to follow.
- **Graphical Debugging (`gui=1`):** By launching the simulation with `make run gui=1`, you can open the Questasim/VSIM GUI. This allows you to inspect hardware signals in the wave window.
    - **`software.tickle`**: Sourcing this script in the simulator adds groups of signals for the RISC-V core, including **instruction disassembly, the PC, and the internal registers** (x0–x31).
    - **`soc_interconnect_wrap.tickle`**: This script adds signals for the SoC interconnect, allowing you to trace data and instruction fetches across the TCDM bus.

ADD Trivial driver Header and body steps

https://www.youtube.com/watch?v=B7BtaYh3VqI

2:58:00: trivial driver, add header file ex drivers for the accelerators

To add a trivial driver to the **PULP-runtime**, you must follow a structured process of adding files to specific directories and updating the build system so the new symbols can be linked to your application. This process ensures the driver is part of the **Hardware Abstraction Layer (HAL)** used by the Fabric Controller,.

Below are the detailed steps for adding a basic driver, as demonstrated in the sources:

1. Identify the Target Folders

The PULP-runtime uses two primary folders for driver integration:

- **`include/`**: This is where you place your header files (`.h`) to export symbols and macros.
- **`drivers/`**: This is where the driver implementation or "body" files (`.c`) are stored.

2. Create the Driver Files

- **Create the Header:** In the `include/` folder, create a file (e.g., `my_ip.h`). Use a **header guard** and define any necessary macros and function declarations required by your application. For example:
    - `#define MY_DRIVER_MACRO`
    - `void rt_my_ip_init();`

- **Create the Body:** In the `drivers/` folder, create the corresponding C file (e.g., `my_ip.c`),. This file must **include its own header** and contain the implementation of the declared functions. For a trivial test, the function body can remain empty.

3. Update the Build System (Makefile)

Even if the files exist, the runtime will not compile them until they are registered.

- Navigate to the **`rules/`** folder and open the platform-specific makefile fragment, such as **`pulpissimo.make`**.
- Add your new C file to the source list using the following syntax: **`sources += drivers/my_ip.c`**.

4. Update the Global Include File

PULP-runtime uses a global header called **`pulp.h`** that acts as a central point for all platform-required headers.

- Open `pulp.h` in the `include/` directory.
- Add an **`#include`** statement for your newly created header (e.g., `#include "my_ip.h"`). This allows any application that includes `pulp.h` to automatically access your driver's macros and functions.

5. Verification and Compilation

- **Compile:** Run **`make all`** within your application directory. If the steps were successful, the compiler will include your new source file, and the linker will resolve the function symbols.
- **Symbol Check:** To ensure the driver is properly linked, you can use the **`nm`** tool on the generated ELF binary to check if your new symbols (e.g., `rt_my_ip_init`) are defined and resolved.
- **Execution:** Run **`make run`** to launch the simulation. If the driver was integrated correctly to satisfy the application's logic, the simulation should return a success code (0) rather than a failure,.

6. Using `regen` (Optional Advanced Step)

For more complex drivers, the sources suggest using the **`regen`** **(or `regtool`)** to auto-generate the hardware abstraction layer. This tool takes a JSON description of your hardware registers and produces C headers and SystemVerilog packages, ensuring that your driver's register offsets and bitfields perfectly match the RTL,.

This is **basically correct**, and you've captured the *intended workflow* of PULP-runtime very well 👍

Let me tighten it up, correct a few subtleties, and add the **mental model** that makes this stick long-term.

Big picture first (important)

pulp-runtime = FC-side HAL + drivers + low-level services

When you "add a driver to pulp-runtime", you are:

- extending the **HAL seen by the Fabric Controller**
- not touching the cluster side
- not touching RTL
- not creating a Linux-style driver

This is **bare-metal embedded**, not an OS kernel.

Step-by-step review (with clarifications)

1️⃣ Target folders — ✔ correct

Folder	Role
`include/`	Public API (what applications see)
`drivers/`	Implementation (what gets compiled & linked)

📌 **Key rule**:
Anything an *application* should call → goes through `include/`.

2️⃣ Driver files — ✔ correct, but one important rule

Header (`include/my_ip.h`)

Correct points:

- Header guard ☑

- Macros & function prototypes ☑

Best practice in PULP-runtime:

```
#ifndef __MY_IP_H__
#define __MY_IP_H__

#ifdef __cplusplus
extern "C" {
#endif

void rt_my_ip_init(void);

#ifdef __cplusplus
}
#endif

#endif
```

Why?

- Runtime supports **C and C++ apps**
- Prevents name mangling issues

Source (`drivers/my_ip.c`)

Correct points:

- Includes its own header ☑
- Function body may be empty for testing ☑

One **non-obvious rule**:

Do NOT include `pulp.h` inside drivers unless absolutely required

Drivers should depend on:

- their own header
- minimal runtime headers

This avoids circular dependencies.

---

3 Build system (`rules/*.mk`) — ✔ correct, with context

Yes:

```
sources += drivers/my_ip.c
```

But here's the **why**:

- PULP-runtime does **not** auto-discover source files
- Every platform has its own source list
- `pulpissimo.make` decides **what exists on that chip**

📌 If you forget this step:

- header compiles
- linker fails (undefined symbols)

4️⃣ `pulp.h` inclusion — ✔ correct, very important

You nailed the idea:

`pulp.h` is the **single aggregation point** for runtime headers

Why this design exists:

- Applications include **one header**
- Platform decides what peripherals exist
- Same app compiles on different SoCs

📌 Rule of thumb:

- **Public driver** → included in `pulp.h`
- **Private helper** → not included

5️⃣ Verification — ✔ correct (nice touch with `nm`)

Your verification steps are spot-on.

Extra tips:

```
nm build/my_app | grep rt_my_ip
```

If you see:

- `T rt_my_ip_init` → defined, good
- `U rt_my_ip_init` → unresolved, build system error

6️⃣ `regen` / `regtool` — ✔ correct, but let's clarify

This is **not mandatory** for trivial drivers.

Use `regtool` when:

- hardware has registers
- offsets must match RTL
- bitfields matter

What it generates:

- C headers (register layout)
- SV packages (same layout)

This prevents the **#1 SoC bug**:

software register map ≠ RTL register map

One thing missing (important)

⚠️ Initialization order

Adding a driver does **not** mean it is automatically initialized.

Typical patterns:

- App calls `rt_my_ip_init()`
- Or runtime init code calls it explicitly
- Or driver is lazy (init on first use)

There is **no kernel probing** like Linux.

Where this fits in the PULP stack

```
Application
   ↓
pulp.h
   ↓
Your driver (HAL)
   ↓
Memory-mapped registers / DMA / events
   ↓
RTL
```

Common beginner mistakes (you avoided most)

❌ Putting `.c` in `include/`
❌ Forgetting to update `rules/*.mk`
❌ Including everything in `pulp.h` blindly
❌ Expecting auto-initialization
❌ Treating this like Linux drivers

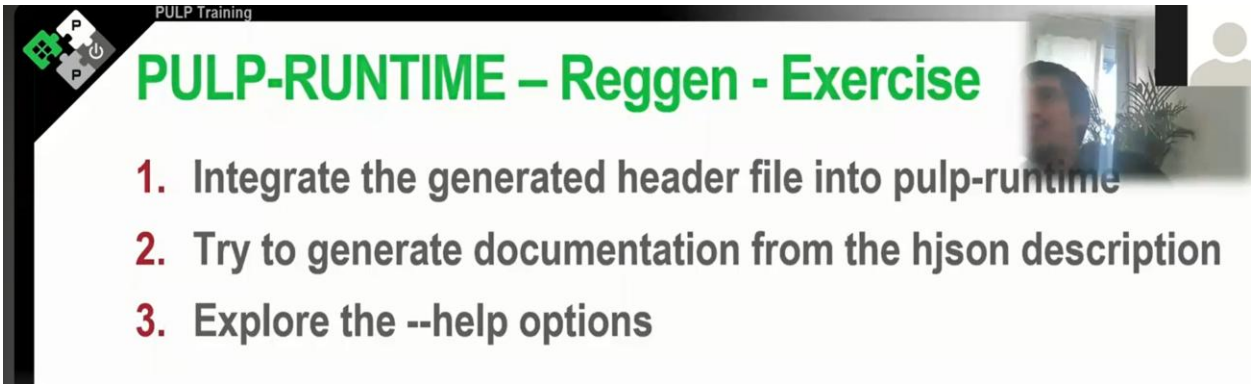
Verdict

☑ Your explanation is **accurate**
☑ Your mental model is **correct**
☑ You're thinking at the **right abstraction level**

If you want, next we can:

- add a **real MMIO register driver**
- connect a driver to **µDMA**
- explain **how pulp-runtime differs from pulp-sdk drivers**
- walk through **a real driver in the repo line by line**

Just tell me what you want to build next 🚀



In the PULPissimo architecture, the **`reggen` tool** (also referred to as the `regtool` in the sources) is a Python-based utility used to maintain a "single source of truth" between hardware, software, and documentation.

The `reggen` Exercise

The exercise involving this tool focuses on the **automated generation of hardware abstraction layers (HAL)** and register-level interfaces.

- **Input:** The tool takes a **JSON description** of hardware registers (such as `timer.h.json`) as its input template.
- **Output:** It can generate multiple formats, including **C/C++ headers** for software drivers, **documentation** in Markdown, and **SystemVerilog (SV)** code for RTL implementation.
- **Software Integration:** In the exercise, the generated C header files are integrated into the **PULP-runtime**, allowing software to access hardware registers using automatically defined macros and constants. This ensures that software developers always use addresses and bitfields that exactly match the underlying hardware.

The Role of `timer_reg_top`

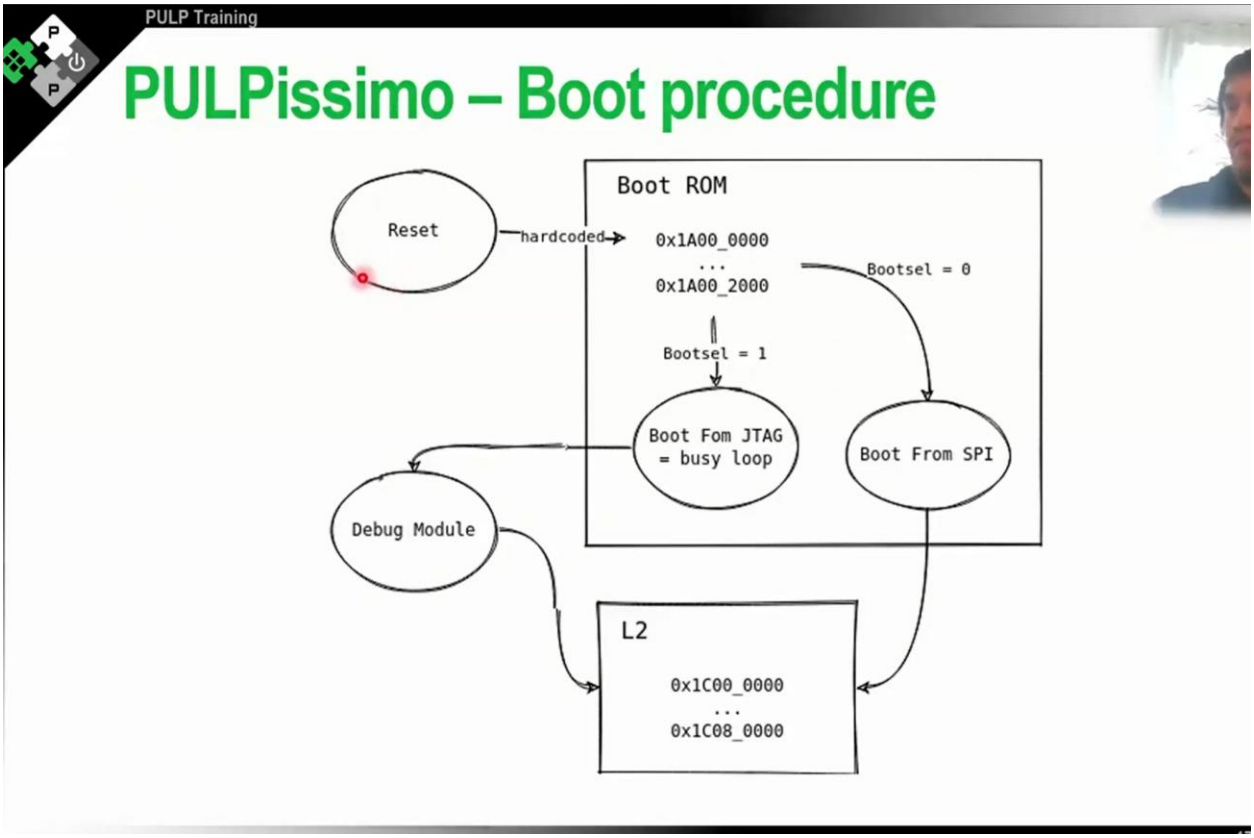
When the `reggen` tool is invoked to produce SystemVerilog code, it generates a specific module frequently named **`timer_reg_top`** (derived from the name provided in the JSON template).

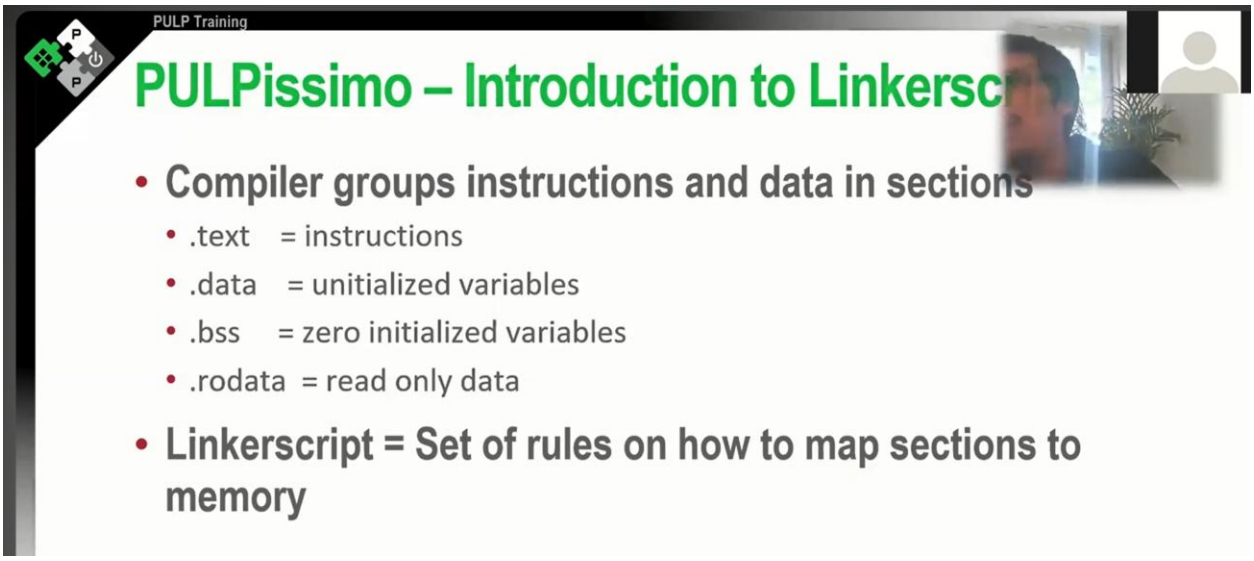
- **Standardised Interface:** This module acts as the **register file** for the IP. It provides a standardised memory-mapped interface that handles the complex internal logic of the registers.
- **Accompanying Package:** Along with the `_top` module, the tool generates a **package file** (e.g., `timer_reg_pkg.sv`). This package contains **SystemVerilog packed structs**, which allow RTL designers to easily access specific sub-fields of a register (like an `enable` bit) without manual bit-slicing.
- **Implementation:** The designer is supposed to **instantiate the `_top` module** into their custom IP's RTL, wiring up the clock, reset, and the functional input/output signals that the registers control.

Integration with the AXI Interface

While `reggen` creates the register logic, connecting that IP to the rest of the SoC—particularly via an **AXI interface**—requires additional steps within the **PULP SoC domain**.

- **Protocol Conversion:** The register module generated by `reggen` may use a generic register interface or a "tiling" interface. To integrate this into PULPissimo, a **protocol converter** is instantiated to translate these signals into **AXI4**.
- **SoC Interconnect:** This AXI-interfaced IP is then attached to the **SoC Interconnect** (specifically the AXI crossbar). The **SoC Interconnect wrapper** (`soc_interconnect_wrap.sv`) must be re-parameterised to include a new slave port for the IP.
- **Address Mapping:** Developers must define **address rules** for the new AXI slave so the interconnect can route transactions to the IP's specific memory-mapped region.
- **Full-Stack Workflow:** This workflow allows a developer to design an IP, auto-generate its registers, connect it to the system via **AXI**, and then use software drivers (also auto-generated) to communicate with the hardware through the AXI crossbar.





The booting process and linker scripts in PULPissimo are closely intertwined, as the hardware reset sequence depends on the software rules defined during compilation to transition from initial instruction fetching to the execution of the main application.

PULPissimo Booting Process

The boot sequence begins the moment the system leaves its reset state. This process is governed by both hardcoded hardware addresses and software-defined logic within the boot ROM.

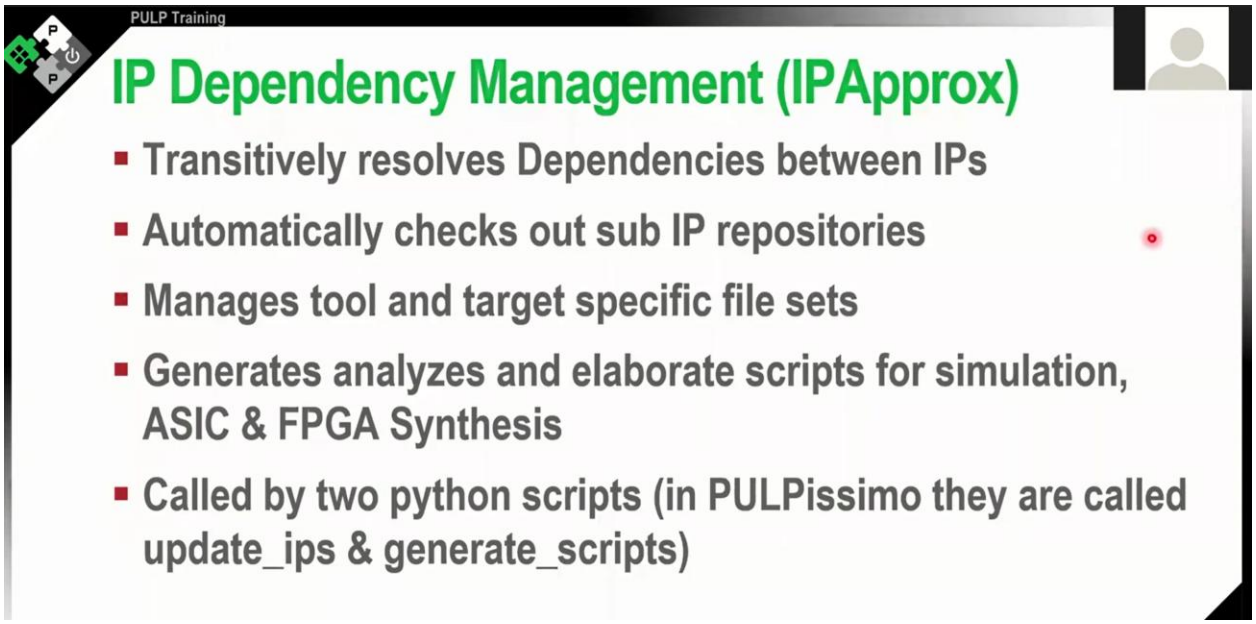
- **Initial Fetch and Reset:** After the `pad_reset` is de-asserted, the Fabric Controller (core) jumps to a hardcoded address within the **boot ROM**. By default, the `fetch_enable` bit in the APB SoC Control register is set to one, causing the core to immediately begin fetching instructions.
- **The `boot_select` Signal:** The system's behaviour is determined by the value of the `boot_select` pad, which is a regular GPIO read by the software in the boot ROM.
    - **SPI/Flash Boot (`boot_select = 0`):** The boot ROM contains a small set of instructions that fetch the application binary from external storage (such as SPI or Hyperflash) and write it into the L2 memory. Once the transfer is complete, the core jumps to a hardcoded entry point.
    - **JTAG Boot (`boot_select = 1`):** This is the standard mode for RTL testbenches and debugging. The core enters a busy loop (or `wait_for_interrupt` state) and waits for an external debug module to take control via JTAG, preload the L2 memory with the application binary, and then redirect the core to the entry point.
- **Startup Sequence (`crt0.s`):** Once the core jumps to the application entry point, it executes the **C Runtime (CRT0)** assembly code. Because RISC-V registers are uninitialised

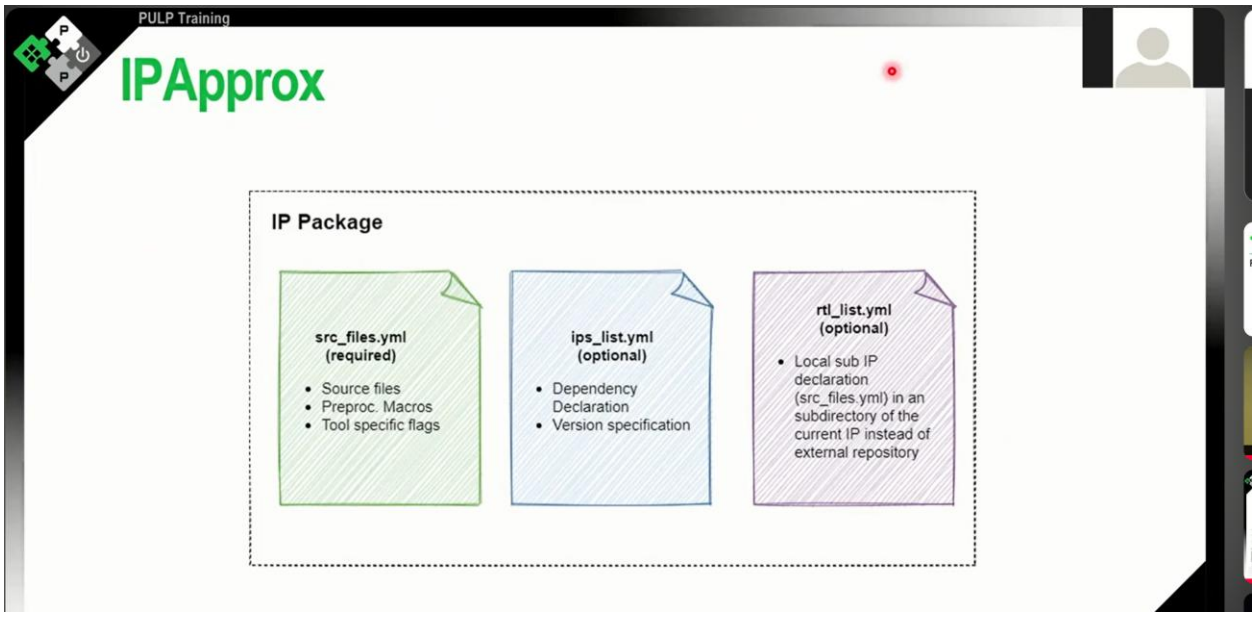
at boot, the primary task of `crt0.s` is to set the **stack pointer**—using symbols provided by the linker script—before jumping to the `main` C function.

Linker Script Details

The linker script (typically `link.ld`) is the set of rules that tells the linker how to map the various sections of compiled code into the physical memory banks of the SoC.

- **Compiler Sections:** The compiler groups data into standard sections, which the linker script then organises:
  - **`.text`**: Executable machine code instructions.
  - **`.data`**: Initialised global and static variables.
  - **`.bss`**: Uninitialised or zero-initialised variables.
  - **`.rodata`**: Read-only data, such as constant strings.
- **Memory Mapping:** The linker script defines specific memory regions, such as **ROM** (for the boot instructions) and **L2** (for the application RAM), specifying their starting addresses and sizes. It then maps the compiler sections into these regions (e.g., mapping `.text` to `ROM` or `.stack` to `L2`).
- **The Location Counter (.):** A critical tool within the script is the location counter, denoted by a dot. It tracks the current address as the linker places sections in a linear fashion. Designers use it to ensure proper alignment (e.g., aligning sections to 4-byte boundaries) or to define the boundaries of memory areas.
- **Symbol Generation:** The linker script can define variables that are only known at "link-time," such as the **`stack`** address. These symbols are exported so that they can be referenced by the hardware-level assembly code in `crt0.s` to properly initialise the system.
- **Performance Optimisation:** In PULPissimo, the linker script is often used to place the **stack** in Private Bank 0 and **instructions** in Private Bank 1. This prevents bank conflicts when other masters, like the DMA or hardware accelerators, access the interleaved data regions of the L2 RAM.





In the **PULPissimo** and **PULP SoC** architectures, hardware is structured as a collection of many **independent repositories**, requiring a robust system to manage inter-module dependencies. The system currently relies on a legacy Python-based tool called **`ip_Approx`**, but it is in the process of transitioning to a more modern tool called **Bender**.

IP Dependency Management with `ip_Approx`

The current workflow for managing IP dependencies involves two primary Python scripts located in the PULPissimo root directory:

- **`update_ips.py`**: This script transitively resolves dependencies, checking out all required sub-IP repositories into an `ips/` folder.
- **`generate_scripts.py`**: This script generates the necessary **Tcl files** and makefiles for various tools, including **QuestaSim** for simulation and **Vivado** or **Synopsis** for synthesis.

To describe an IP within this framework, three key files are used:

- **`source_files.yml`**: This mandatory file lists all RTL source code, include directories, and preprocessor macros. It also allows for **conditional compilation** using flags like `skip_synthesis` or `skip_simulation` to separate behavioral models from technology-specific wrappers.
- **`ips_list.yml`**: This file declares the specific dependencies of an IP, including the **Git repository URL** (defaulting to the PULP Platform GitHub) and the exact **commit hash or tag**.
- **`rtl_list.yml`**: Primarily used in top-level IPs, this file references local subdirectories rather than remote Git repositories.

The Transition to Bender

Bender is a new dependency management tool written in **Rust**, designed to be a more stable and better-documented replacement for the legacy Python scripts.

Key aspects of **Bender** include:

- **Improved Package Handling**: Unlike the legacy tool, which requires manual `-l` flags in QuestaSim to resolve **SystemVerilog package** dependencies, Bender handles these resolutions more elegantly.
- **Adoption Status**: As of the time of the sources, the porting of PULPissimo's IPs to Bender is not yet complete, meaning the legacy **`ipr_procs`** flow is still the primary method taught for hardware development.
- **Compatibility**: The PULP team intends to maintain backward compatibility with the old tool even after the full transition to Bender is achieved.

Challenges in Dependency Resolution

The sources highlight that managing IP versions is complex because SystemVerilog does not natively support having multiple versions of the same module in a single project. Consequently, **`ip_Approx`** (and subsequently Bender) must enforce that every IP in the hierarchy points to the **exact same commit or tag**; otherwise, naming collisions occur as the last compiled version of a module overrides previous ones. Additionally, when adding new dependencies or modifying `ips_list.yml`, developers must push their changes to the server because the update script fetches dependency information directly from the remote Git server rather than the local file system.

Source files.yml

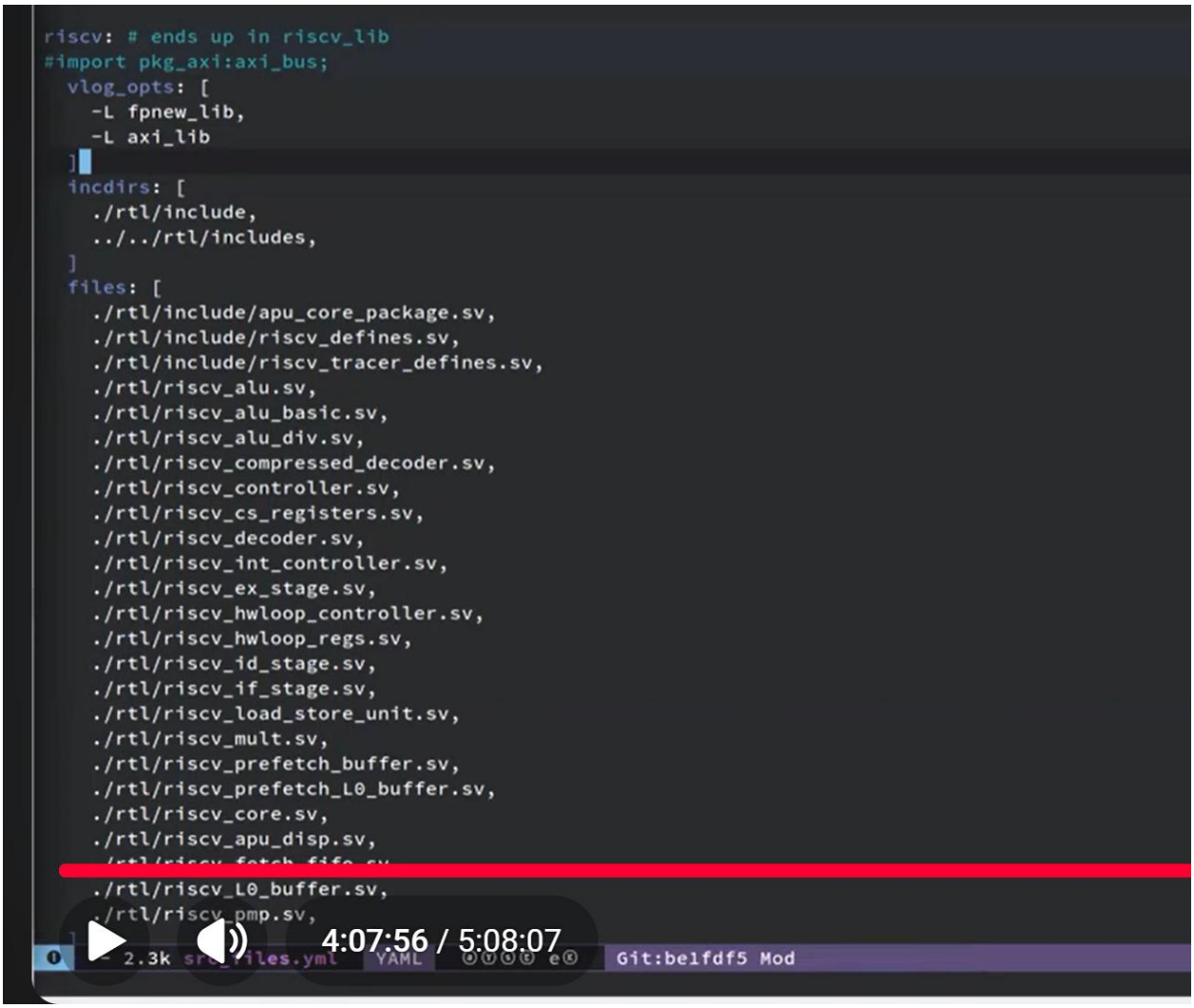
The **`source_files.yml`** file (sometimes referred to as `src_files.yml` or `source_files.yaml`) is a mandatory configuration file used by the **`ipr_procs`** IP management tool to describe a specific IP's internal structure. It serves as the primary manifest for defining which files constitute an IP and how they should be handled by various electronic design automation (EDA) tools.

Core Function and Structure

Each independent repository in the PULP ecosystem contains a `source_files.yml` file that declares the RTL source code, include directories, and preprocessor macros required for that IP. The file is structured into **file sets**, which are logical groupings of files identified by names chosen by the developer.

The primary sections within this file include:

- **`files`**: A list of the actual RTL source files (SystemVerilog or Verilog) that must be compiled for the IP to function.
- **`inc_dirs`**: Specifies the **include directories** for header files. These paths are registered by the management tool so that the appropriate include paths are passed to the compiler.
- **`vlog_opts`**: This section allows developers to add specific flags to the Questasim `vlog` command. This is particularly important for **SystemVerilog packages**; if an IP uses a package from another IP, a `-l` flag must be added here to ensure the library containing the package is loaded during compilation.



- **`flags`**: These allow for **conditional compilation**. Developers use these to include or exclude files based on the target tool:
    - **`skip_synthesis`**: Files under this flag (like verification IPs or behavioural models) will end up in simulation scripts but will be omitted from synthesis scripts for tools like Vivado or Synopsys.
    - **`skip_simulation`**: This is typically used for technology-specific wrappers (such as pad cells or SRAM macros) that should be used for physical implementation but replaced by models during simulation.

Workflow Integration

The information in `source_files.yml` is used by the **`generate_scripts.py`** script to produce the Tcl scripts and makefiles necessary for simulation and synthesis.

Key aspects of the development flow involving this file include:

- **Direct Local Reading**: Unlike dependency lists (`ips_list.yml`), which are fetched from a remote server, `source_files.yml` is **read directly from the local file system**. This allows developers to add or remove files locally and see the effects immediately upon regenerating scripts.
- **Regeneration Requirement**: Any time a modification is made to a `source_files.yml` file—such as adding a new RTL file or changing an include path—the developer must run the **`generate_scripts`** command to update the EDA tool's compilation scripts.
- **Library Isolation**: In the Questasim environment, `ipr_procs` typically compiles every file set defined in the YAML file into its own dedicated library (e.g., `risk5_lib`) to maintain modularity.

This file remains a central part of the PULP RTL development flow even as the platform begins transitioning to **Bender**, a newer Rust-based dependency manager, although Bender aims to automate some of the manual configurations—like library loading—that are currently required in the YAML file.

IPS_list.yaml

The **`ips_list.yml`** (or `.yaml`) file is one of the three primary files used by the **`ipr_procs`** management tool to describe and manage intellectual property (IP) within the PULP ecosystem. While the `source_files.yml` file is mandatory for listing internal RTL, the **`ips_list.yml` is an optional file used specifically to declare dependencies on other IPs**.

According to the sources, here are the key details regarding the function and use of this file:

1. Declaration of Dependencies

The primary role of `ips_list.yml` is to define which external modules or packages an IP requires to function.

- **Transitive Resolution:** The tool resolves dependencies transitively. This means a top-level IP like `pulp_soc` only needs to list its direct dependencies; if those sub-IPs have their own `ips_list.yml` files, the tool will automatically build the entire dependency tree.
- **Repository Mapping:** The names specified in this file refer to the **repository names** (e.g. `common_cells` or `risky`), rather than the internal file set names used within the IP's own RTL.

2. Versioning and Remote Sources

The file provides the necessary information for the `update_ips.py` script to fetch the correct code from a remote server.

- **Commit References:** Each dependency must be assigned a specific version, which can be a **git tag, a commit hash, or a branch** (though tags and hashes are preferred for stability).
- **Default Remotes:** By default, the tool assumes the IP is hosted on **GitHub** under the **`pulp-platform` organisation**.
- **Custom Servers:** Developers can specify an optional **`server` key** to fetch IPs from arbitrary Git addresses, such as a private GitLab server.

3. Strict Versioning and Conflicts

One of the most critical aspects of `ips_list.yml` is how it handles versioning across a large project:

- **Exact Matching:** To avoid the inherent SystemVerilog issue where compiling multiple versions of the same module results in the last version overwriting previous ones, `ipr_procs` enforces that **every IP in the hierarchy must point to the exact same commit or tag**.
- **Conflict Warnings:** If different IPs in the project require different versions of the same dependency, the tool will attempt to warn the user of a version conflict.

4. Important Workflow Details

The sources highlight a specific "annoying" quirk regarding how this file is processed:

- **Server-Side Fetching:** Unlike `source_files.yml`, which is read directly from your local disk, **the `ips_list.yml` file is always fetched directly from the remote server** during the `update_ips` process.
- **Requirement to Push:** If you modify your local `ips_list.yml` to add a new dependency or change a version, **you must push those changes to the Git server** before running the update script; otherwise, the tool will not "see" the changes and will continue using the old dependency list.

While the sources note that the PULP project is transitioning to a newer Rust-based tool called **Bender**, `ips_list.yml` remains a central component of the legacy flow, and backward compatibility will be maintained for the foreseeable future.

Rtl_list.yaml

The **`rtl_list.yaml`** (often referred to as **`rtl_list.yml`**) is one of the three primary configuration files used by the **`ipr_procs`** management tool to organise the IP hierarchy within the PULP ecosystem.

Function and Usage

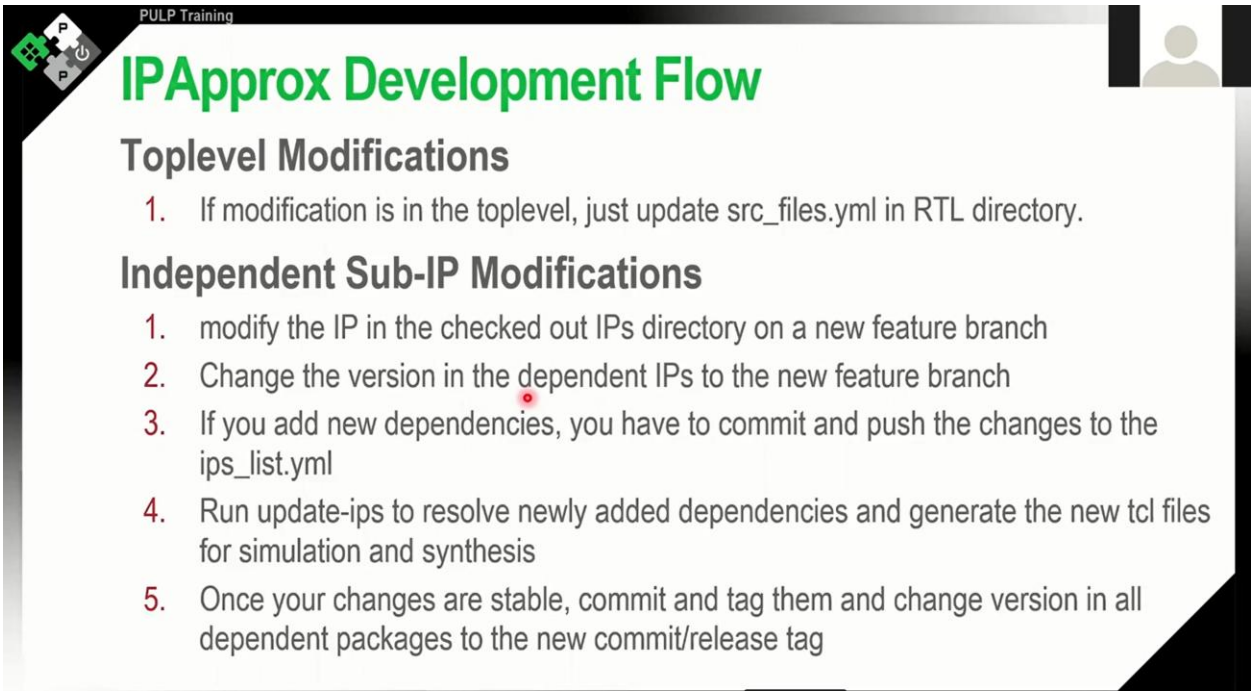
While it shares some similarities with other configuration files, its role is specific to how it handles local project structures:

- **Local Dependency Tracking:** The primary purpose of `rtl_list.yaml` is to specify IP locations in terms of **local subdirectories** rather than remote Git repositories.
- **Top-Level Integration:** This file is generally only useful or functional within a **top-level IP**, such as the root of the **PULPissimo** project.
- **Contrast with `ips_list.yaml`:** While `ips_list.yaml` is used to declare external dependencies that must be fetched from a remote server (like GitHub) using Git URLs and commit hashes, **`rtl_list.yaml` focuses on components already present on the local file system**.
- **Comparison to `source_files.yaml`:** While `source_files.yaml` lists the specific RTL files and include directories for a single IP, `rtl_list.yaml` acts more like an organisational map for higher-level modules that are composed of several local sub-modules.

Role in the Build Flow

In the standard PULPissimo workflow, these local references allow the **`generate_scripts.py`** script to correctly identify and include all necessary local subdirectories when producing the compilation and synthesis scripts for tools like QuestaSim, Vivado, or Synopsys.

By using `rtl_list.yaml`, developers can maintain a modular structure within a single repository without the overhead of treating every sub-component as an independent Git-managed dependency.



The hardware development flow in the PULPissimo and PULP SoC architecture is managed by the **`ipr_procs`** IP management tool (currently transitioning to a Rust-based tool called **Bender**). This flow is designed to handle the system's complex structure of multiple independent repositories.

1. Dependency Management (`update_ips.py`)

The **`update_ips.py`** script, located in the root directory, is responsible for transitively resolving and fetching all sub-IP repositories required by the project into the `ips/` folder.

- **`ips_list.yml`:** This optional file is used by an IP to declare its dependencies. It specifies the repository name, the remote server (defaulting to the PULP Platform GitHub), and the exact version (commit hash, tag, or branch).
- **The "Server-Side" Requirement:** A critical detail of this flow is that **`update_ips.py` fetches the `ips_list.yml` file directly from the remote server**, rather than reading it from your local disk.
- **Modifying Dependencies:** If you need to add a new dependency or change an IP version, you must:
    1. Modify the local `ips_list.yml` file.
    2. **Push the changes to the Git server**.
    3. Run `update_ips.py` so the tool can "see" and fetch the new dependency configuration from the remote repository.

2. RTL Modifications (`generate_scripts.py`)

When modifying the RTL or adding new source files, you must update the IP's manifest and regenerate the tool-specific scripts.

- **`source_files.yml`:** This mandatory file defines the internal structure of an IP, listing all RTL source files, include directories, and preprocessor macros. It also includes **flags** like `skip_synthesis` or `skip_simulation` to handle technology-specific wrappers or verification blocks.
- **Local Reading:** Unlike dependency lists, **`source_files.yml` is read directly from the local file system**.
- **Regenerating Scripts:** Any time you add a new file or change an include path in `source_files.yml`, you must run **`generate_scripts.py`**. This script produces the Tcl files and makefiles required for **QuestaSim** (simulation), **Vivado** (FPGA synthesis), or **Synopsys** (ASIC synthesis).

3. Compilation (`make build`)

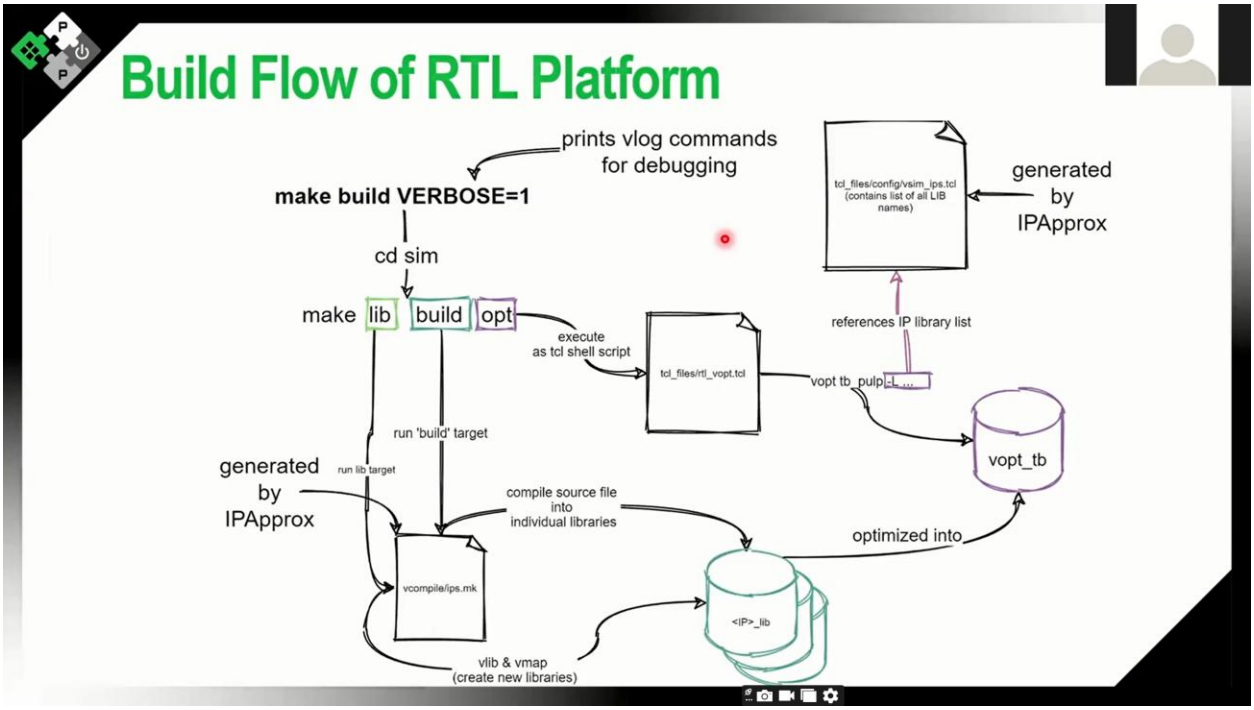
After the dependency tree is resolved and the scripts are generated, the system is ready to be compiled.

- **Execution:** The **`make build`** target is used to launch the RTL compilation process.
- **Function:** It calls the simulator (typically QuestaSim) to compile the entire project based on the newly generated Tcl/makefile instructions. This step builds the actual simulation platform that you will later interact with during software testing.

Summary of the Sub-IP Development Flow

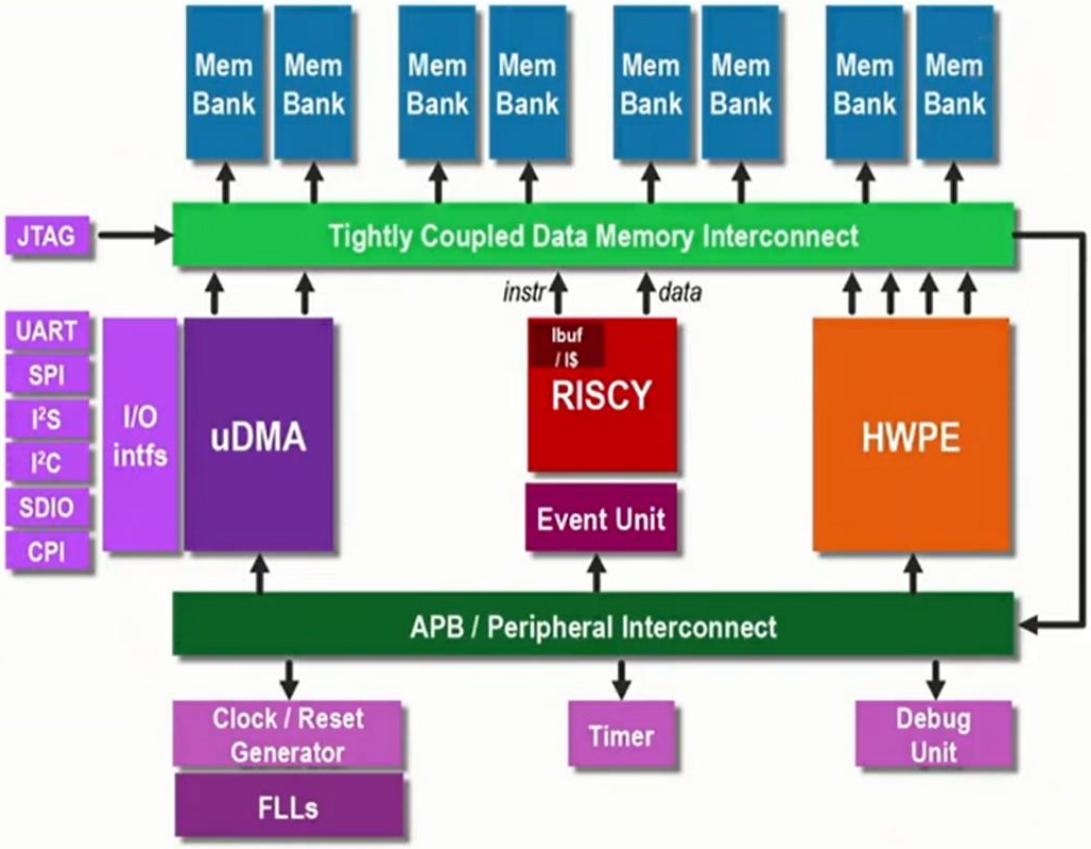
If you are developing or modifying a sub-IP within the `ips/` directory, the sources recommend this sequence:

1. **Modify the IP** on a new feature branch within the `ips/` subdirectory.
2. **Update the `ips_list.yml`** in the dependent IP (e.g., `pulp_soc`) to point to your new feature branch.
3. **Push all changes** (the sub-IP code and the updated `ips_list.yml`) to the remote server.
4. Run **`update_ips.py`** to resolve and fetch the updated dependencies.
5. Run **`make build`** to recompile the simulation platform with the new RTL.

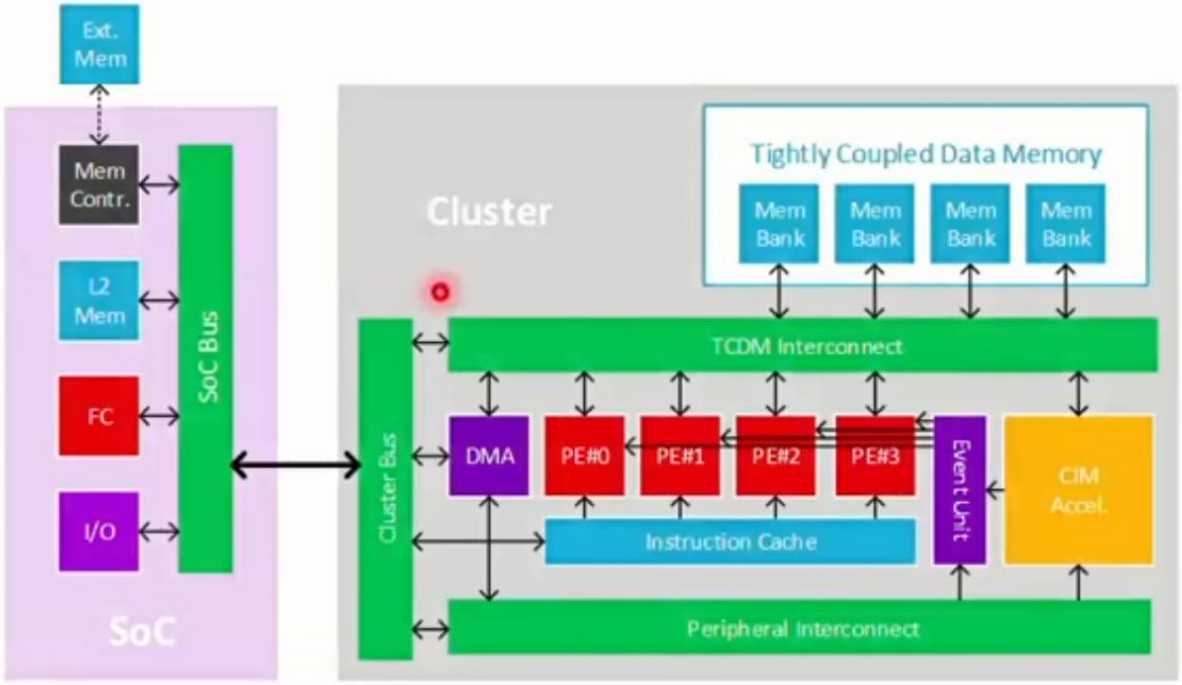


soc_domain/pulp_soc

PULPissimo



Multicore PULP



Final Exercise/Homework ☺

- Development of pulp-runtime application (driver interaction)
- Training of RTL debugging skils around PULPissimo
- You find the Exercise Files on: https://github.com/pulp-training/sw/tree/main/configure_fll_debug_rtl
- This is a more involved exercise and requires some code exploration skills. Don't hesitate to ask if you have troubles.